C-TreePO: Audited Compression Trees for Long-Document Measurement

Mitchell Linegar*

Abstract

Measurement on long documents is hard because hierarchical summarization can destroy the task signal before the judgment is ever made. We introduce C-TreePO (Compression-Tree Preference Optimization), which treats recursive summarization as a learned mergeable sketch: split the document into contiguous leaves, summarize each leaf, merge sibling summaries pairwise up a tree, and judge the root summary in place of the document. Three local laws (leaf sufficiency, idempotence, and merge consistency) state what mergeability means for a learned summarizer: every node summary must receive the same task-oracle value as the raw span below it, and an induction carries that consistency from leaves to root. Labels, rewards, or preferences that depend on the document only through that value can then be collected on summaries instead of raw documents. Most judgments social scientists care about can be expressed as preferences between documents, so preference optimization over long documents becomes preference optimization over their compression trees. The laws are also observable: realized nodes can be sampled with logged inclusion probabilities and their raw spans scored alongside their summaries. Design-based supervised learning then applies twice, once to align an approximate oracle's labels with the oracle and once to align node summaries with those approximate labels, giving design-based estimates of violation rates and mean oracle distortion. We use party-position measurement from Manifesto Project text as a running example, substituting document-level observations for node-level annotation: a prompt-only C-Tree over one open-weight model agrees with expert surveys as closely as the same model reading each document whole (correlation averaged over six policy dimensions, 0.787 versus 0.771 for the uncompressed read), and the agreement is nearly constant as leaves grow from about a paragraph to tens of pages. Hand-coded quasi-sentence labels then induce an oracle value at every tree node; this census validates the sampled audit and shows that learned merges compose where the coded signal is dense and fail where it runs thin.

1 Introduction

Researchers who study long documents (party manifestos, legislative records, clinical narratives) routinely need a single judgment that depends on information spread across the entire text. An expert left-right score for a manifesto reflects how environmental, economic, and social positions combine; a severity rating for a clinical narrative depends on symptoms reported pages apart. Producing these judgments at scale requires either expensive full-document annotation or some form of compression. Compression is also exactly what can break the measurement: a summarizer built without the judgment in mind discards the part of the text the judgment needed. C-TreePO (Compression-Tree Preference Optimization) provides the compression and proves when it is safe.

The idea is simple. Split the document into contiguous leaves (a fixed partition, for now), summarize each leaf, merge sibling summaries pairwise up a binary tree, and judge the root

*California Institute of Technology and Washington University in St. Louis. Email: mitchelllinegar@gmail.com.

summary in place of the document. If the summaries preserve the right information at each step, the root judgment is as good as reading the full document.

We formalize "the right information" as one invariant and three local laws. Call the judgment the *task oracle* f^*: the value an expert would assign given unlimited time to read the raw text in full. A node summary is *oracle-consistent* with the raw span below it when both receive the same value from f^*. That is the entire bar a summary has to clear: it may discard anything else; it must keep whatever moves f^*. The summary answers to the judgment, not to the document.

A concrete example shows what can go wrong without the local laws. A platform can be environmentally interventionist and, in the same breath, economically restrained. Summarizing the environment section on its own and the economy section on its own can capture each stance correctly, and merging those two summaries can still lose the fact that the party is more interventionist here and more restrained there. Whichever way that interaction gets rounded off, the manifesto's overall left-right score moves, and the mistake happened at the merge, not at either leaf. The same failure has an exact arithmetic form: when the task is to count how many times a hidden state changes along a sequence, a change that falls on a leaf boundary is invisible to either leaf, and a sum of per-leaf counts systematically undercounts. The merge must carry enough information to answer the question at the boundary.

Three local laws keep the invariant intact, and each one names a place a tree can fail. Leaf sufficiency (C1) starts it: each leaf summary must be oracle-consistent with its raw leaf, so a paragraph compressed into a sentence keeps the clause that would have moved the judgment. Idempotence (C2) protects it under recompression: resummarizing a stored summary must leave its oracle value untouched, so a cleanup pass cannot silently drift. Merge consistency (C3) passes it up one level at a time: the merge of two oracle-consistent child summaries must be oracle-consistent with the parent span, so cross-span evidence survives the split. The guarantee then follows by a simple induction on the tree: two good child summaries make one good parent summary, and repeating that merge triangle from the leaves upward carries the invariant to the root. Section 3 states the induction as one proposition.

The recipe is that of a mergeable sketch from streaming data systems [Agarwal et al., 2013], generalized from hand-designed merge rules to learned summarizers. The three laws restate the sketch algebra at the task oracle: C1 says an encoded block keeps what the query needs, C2 says re-encoding a sketch changes nothing, and C3 says merging two sketches matches sketching the combined input. The generalization changes the checking: algebra proves a classical merge correct once and for all; a learned merge has to be checked in use.

The framework offers three things. First, preservation: when the local laws hold, the root summary supports the same task judgment as the full document. Second, portable supervision: labels, rewards, or preferences that depend on the document only through its oracle value can be collected on summaries or nodes instead of raw documents. Most judgments social scientists care about can be expressed as preferences between documents, so preference optimization over long documents becomes preference optimization over their compression trees. Third, an operational audit: the laws are observable, so a realized node can be sampled and its raw span scored alongside its summary. Sampling with logged inclusion probabilities gives design-based estimates of the violation rate and the mean oracle distortion. Design-based supervised learning applies twice [Egami et al., 2024]: once to align an approximate oracle's labels with the oracle, and once to align node summaries with those approximate labels. The audit turns the induction into an accuracy claim about one particular deployed tree; Section 4 develops it as a survey of the realized nodes.

The laws constrain what a summarizer must do; any operator from text to text can do it. The most capable such operator today is a large language model working from a prompt, and the flagship experiment runs exactly that: a prompt-only C-Tree over one open-weight model, with

no fine-tuning and no architecture beyond the prompts themselves, scoring party positions from Manifesto Project text. Document-level observations stand in for node-level annotation throughout. The tree matches the same model's full-document read (macro Pearson 0.787 versus 0.771, in the band of published single-model baselines [Benoit et al., 2025]), and performance is nearly constant across leaf sizes, moving by 0.027 as leaves grow from about a paragraph to tens of pages. That flatness is what the laws predict: when merges preserve the oracle value, the root score is indifferent to how the document is cut. Quasi-sentence codes then induce an oracle value at every tree node; this census validates the sampled audit and traces where learned merges compose and where sparse constructs fail.

The contributions are four. We formalize a task-relative local-law framework for audited compression trees, built on one invariant (oracle-consistency) and three laws that start, protect, and propagate it. We prove a root-preservation guarantee by induction on the tree and give the condition under which labels, rewards, or preferences move from raw documents to tree summaries or nodes. We give a design-based node audit, with sampled gold-standard labels, logged inclusion probabilities, and a calibration step separating judge error from sampling error, that turns the guarantee into a deployable instrument. We show Manifesto Project evidence at two supervision levels: document-label parity with the same model's full-document read, and node-label results from hand-coded quasi-sentences that trace where learned merges compose and where they fail.

Section 2 defines compression trees and states the three laws, with the merge triangle and the context-compatibility assumption that lets a summary stand in for its span. Section 3 proves what the laws buy: exact preservation when they hold, and a root error bounded by the audited local distortion when they hold approximately. Section 4 builds the audit as a survey of a deployed tree's realized nodes and prices its error terms. Section 5 carries the laws from trained neural operators to prompted language models, which the audit treats identically. Section 6 reports the Manifesto Project evidence, and Section 7 places the framework against long-context reading, retrieval, surrogate labels, and classical sketches.

2 Compression Trees and Local Laws

Section 1 named the invariant: a summary is good when it keeps whatever the judgment needs and drops the rest. The definitions that follow state, relative to a task oracle, what a node's stored summary must satisfy for the tree to trust it.

2.1 Documents and Oracles

Two objects anchor the framework: the document and the judgment made on it. A document is a finite string of text. Write $x \in \mathcal{X}$ for a document and $u \oplus v$ for the concatenation of two spans, so that a document splits into a left half and a right half. The judgment the tree exists to protect is a task oracle: the expert-level answer a rubric, a coder, or a careful read would produce from the full document. Write this as a function f^*.

Definition 1 (Oracle)**.** *An oracle is a function $f^* : \mathcal{X} \to \mathcal{Y}$ mapping a document to the task value a researcher ultimately wants.*

f^* is expensive, not exotic. Scoring a manifesto's left-right position, rating a clinical note's severity, or coding a contract's risk level are all ordinary judgments. The obstacle is that each one takes a full, unhurried read of a long document, and no pipeline can afford that on every input, at every training step, for every candidate summary. The tree approximates f^* cheaply and tracks when the approximation is trustworthy.

A summary is exactly as good as what it keeps of the oracle's judgment. Nothing else about it matters. Formally, a raw span x and a candidate summary s are *oracle-consistent*, written $s \sim x$, when the oracle cannot tell them apart:

$$s \sim x \quad \Longleftrightarrow \quad d_{\mathcal{Y}}(f^*(s), f^*(x)) = 0.$$

In words: s is task-sufficient for x. The summary may compress away style, redundancy, and every detail f^* never looks at; it may not compress away the one clause, transition, or qualifier that would move $f^*(x)$. This is the same task-relative reading a sufficient statistic gets in classical statistics [Fisher, 1922, Blackwell, 1953]. Sufficiency is always sufficiency for something, and here that something is f^*, not the full text distribution. A short summary can be sufficient for one oracle and useless for another. The relation $\sim$ is defined against a fixed f^*, and every claim that follows inherits that dependence. Everything the framework asks of a summary lives in this one relation. The three local laws below demand it at three different calls.

2.2 The Tree

With the oracle and its consistency relation fixed, the computation comes next. Fix a partition of the document into contiguous leaves $b_1, \ldots, b_k$, the same fixed-partition assumption carried from Section 1. Arrange the leaves at the bottom of a binary tree T. Each internal node u owns a contiguous raw span, written $S(u)$: the concatenation of the raw text under u's left and right children. At the leaves $S(u)$ is just the block itself; at the root $S(u)$ is the whole document.

The tree computes one thing at every node: a stored summary s_u that stands in for $S(u)$. A summarizer g produces it, and g only ever sees one of two things: a single leaf block, or the concatenation of two already-computed child summaries.

$$s_u \;=\; \begin{cases} g(b_i) & u \text{ a leaf, block } b_i \\ g(s_{u_L} \oplus s_{u_R}) & u \text{ an internal node with children } u_L, u_R. \end{cases}$$

Read bottom-up: summarize each leaf once, then repeatedly summarize the concatenation of two already-summarized children, until one summary sits at the root. g never reads the raw span $S(u)$ at an internal node, only the two child summaries already on hand. Every call therefore fits in a bounded context window no matter how long the document is. The tree is fixed before any summarization begins. The leaf boundaries are chosen once and held constant while the summaries change. A readout f (which may be f^* itself, or a separate accessible judge) turns the root summary into the reported task value. $S(u)$ is a bookkeeping device. Nothing in the pipeline stores it. It exists so a check run after the fact can ask whether s_u still tracks it.

2.3 Mergeable Sketches

The tree recursion is decades old. Streaming systems combine per-shard state into a bounded root answer as mergeable summaries [Agarwal et al., 2013]: hash a data shard into a compact state, merge two states with a fixed operator, and read a query off the merged result at the root, never holding the full stream in memory. C-TreePO is the same recursion with the state, the merge, and the readout learned rather than hand-designed. Table 1 makes the correspondence literal.

The one difference that matters runs down the last row. A classical sketch's merge is a closed-form operator (register-wise max, for instance), so a single algebraic proof covers every shard the sketch will ever see. g here is a learned summarizer with no such algebra behind it. Consistency must be checked. Section 4 builds the check: a sampled audit of realized nodes, not a proof from the definition of g.

Classical mergeable sketch	Oracle-consistent compression tree
Data shard	Text leaf b_i
Sketch state	Summary state s_u
Merge operator	Learned merge call $g(s_{u_L} \oplus s_{u_R})$
Query answered at the root	Task oracle f^*
Correctness by algebraic proof	Correctness by sampled audit

Table 1: The classical recursion and its learned counterpart. Each row on the left is a fixed, hand-designed piece; each row on the right is the learned or measured piece that replaces it.

2.4 Context Compatibility

Local consistency climbs the tree only if swapping a summary for its span leaves the oracle's reading of the surrounding text unchanged. Otherwise a summary could pass its own check, shift the parent's oracle value the moment it sits next to its sibling, and defeat every local check at the parent.

Assumption 1 (Context-compatible oracle)**.** *For every context $w \in \mathcal{X}$, $u \sim v$ implies $w \oplus u \sim w \oplus v$ and $u \oplus w \sim v \oplus w$.*

This is the one piece of glue the induction needs: an oracle-consistent replacement, on either side of a concatenation, leaves the whole span's oracle value where it was. Content-scoring tasks (a position score, a topic count, a risk rating) satisfy it comfortably: the oracle reads for content that survives substitution. Tasks that hinge on discourse order or rhetorical contrast (irony that depends on a specific juxtaposition, for instance) are the ones to check case by case. The assumption can fail there even when every individual summary looks correct in isolation.

2.5 The Three Local Laws

The three local laws make one demand: the string g returns is oracle-consistent with the string g received. They differ only in which realized call is under the microscope. Table 2 states the three.

Law	Formal form	Plain test	Audit unit
C1: Leaf sufficiency	$g(b) \sim b$	Did the leaf summary keep the evidence the task needs?	Leaf nodes
C2: Idempotence	$g(s) \sim s$ for $s \in \text{range}(g)$	Does a cleanup pass on a stored summary drift?	Recompression calls
C3: Merge consistency	$u \oplus v \sim g(u \oplus v) \sim g(g(u) \oplus g(v))$	Did cross-span evidence survive the split into two children?	Merge nodes

Table 2: The three local laws. Each makes the same demand at a different call: the output of a realized call must be oracle-consistent with its input.

C1 (leaf sufficiency) starts the invariant: each leaf summary is oracle-consistent with its raw leaf. The test is whether the leaf summary kept the evidence the task needs; the audit unit is leaf nodes. C2 (idempotence) protects the invariant under recompression: resummarizing a stored summary preserves its oracle value. The test is whether a cleanup pass drifts; the audit unit is recompression calls. C3 (merge consistency) passes the invariant up one level: the merge of two oracle-consistent child summaries is oracle-consistent with the parent span. The test is whether cross-span evidence survived the split; the audit unit is merge nodes.

C1 and C3 carry a summary from the leaves to the root: C1 supplies the base case, C3 every step up. C2 matters exactly when a summary is touched again after it is first computed, for length, for formatting, or for a training pass that revisits an existing node. Two laws move the invariant up the tree once; the third keeps it there.

2.6 The Merge Triangle

C3 is the law that does the climbing, and it is a triangle. Fix a parent span made of two halves, u and v. Three routes describe the whole span, and C3 asks two of them to agree with the third.

$$\underbrace{u \oplus v}_{\text{raw parent}} \quad \sim \quad \underbrace{g(u \oplus v)}_{\text{joint summary}} \quad \sim \quad \underbrace{g\big(g(u) \oplus g(v)\big)}_{\text{tree's route}}.$$

The three vertices are three descriptions of the same span. The raw parent is the span read in full. The joint summary is what one call of g produces from the whole parent at once, a call the tree never makes. The tree's route is the call the tree does make: summarize each half, then merge the two summaries. C3 checks the two links it can reach: raw parent to joint summary, and joint summary to tree's route. The link the tree needs, tree's route to raw parent, follows by transitivity. Figure 1 draws this on a live tree alongside the C1 and C2 checks.

This triangle is the whole engine, and it runs by induction on the tree. The base case is C1: every leaf summary is oracle-consistent with its own leaf. For the inductive step, suppose both children of an internal node already carry oracle-consistent summaries, $s_{u_L} \sim S(u_L)$ and $s_{u_R} \sim S(u_R)$. Context compatibility (Assumption 1) lets each child summary stand in for its raw span inside the parent, so the concatenated child summaries are oracle-consistent with the raw parent span: $s_{u_L} \oplus s_{u_R} \sim S(u_L) \oplus S(u_R)$. C3 at this node says the merge call preserves the oracle value of whatever it receives, so $g(s_{u_L} \oplus s_{u_R}) \sim s_{u_L} \oplus s_{u_R}$. Chaining the two gives $s_u \sim S(u)$, and the parent now plays the role of a child in the next triangle up. Repeat this step at every internal node, and the invariant that started at the leaves reaches the root: the root summary is oracle-consistent with the entire document. Section 3 states this induction as a proposition. The formal version adds only the bookkeeping to state it once for a tree of any shape.

The merge is learned, so the induction is only as good as the premise C3 supplies at each realized node. Section 4 turns that premise into something measurable: a sampled audit of the merges a deployed tree actually makes, estimating how often, and by how much, the triangle fails to close.

3 Guarantees

The laws of Section 2 are local: each one checks a single realized call of g against the input that call saw. The guarantee they buy is global: under those local checks, the summary at the root of the tree preserves the oracle value of the entire document.

Proposition 1 (Oracle-Consistency Propagation)**.** *Fix a document partition $x = b_1 \oplus \ldots \oplus b_k$ and any full binary merge tree T with contiguous leaves. Suppose the realized calls of g on T satisfy C1 (leaf sufficiency) at every leaf and C3 (merge consistency) at every merge node (Table 2), and suppose Assumption 1 (context compatibility) holds. Then the root summary s_{root} is oracle-consistent with the full document:*

$$s_{\text{root}} \sim x, \qquad \textit{i.e.} \qquad d_{\mathcal{Y}}(f^*(s_{\text{root}}), f^*(x)) = 0.$$

If C2 (idempotence) also holds on every realized recompression call, this consistency survives any number of additional summarization passes on the root.

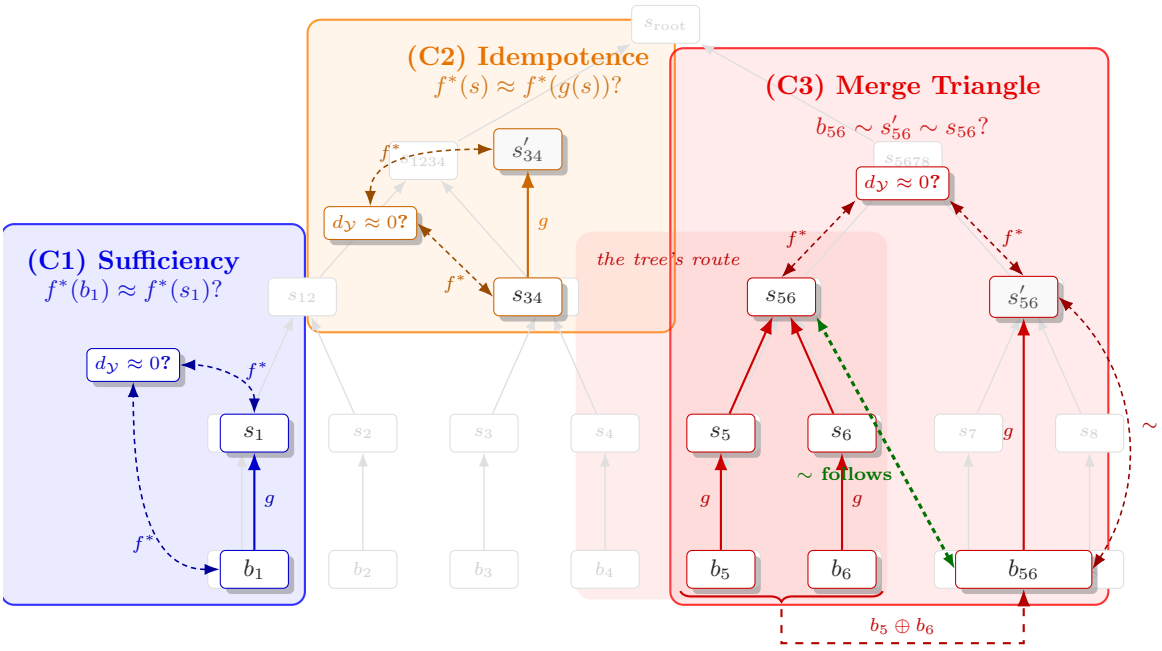


Figure 1: The three local laws on one tree. Each panel highlights the nodes one check touches; gray nodes are not involved. Solid arrows are summarize and merge calls; dotted double-headed arrows are comparisons at the oracle. **C1** (left) compares a leaf summary s_1 to its raw block b_1. **C2** (center) resummarizes a stored summary and checks that the oracle value does not drift. **C3** (right) closes the merge triangle at a realized node. The shaded strip is the tree's route: summarize each half, then merge ($b_5 \to s_5$, $b_6 \to s_6$, then s_{56}). Its three vertices are three versions of the same span: the raw span $b_{56} = b_5 \oplus b_6$ (dashed arrow), its joint summary $s'_{56} = g(b_{56})$, and the tree's summary s_{56}. The audit checks two links, $b_{56} \sim s'_{56}$ and $s'_{56} \sim s_{56}$; the third, the one the tree needs, follows by transitivity (green). Every check fits inside a bounded context window and can be sampled independently.

In plain language: if every leaf summary keeps what its own block needed and every merge keeps what the two halves needed together, the summary at the root is exactly as good as reading the whole document. The proof is a short induction on the tree. C1 anchors the base case at the leaves. At an internal node, the child summaries are oracle-consistent by the induction hypothesis. Context compatibility lets them stand in for the raw children, and C3 closes the triangle between the raw parent, the joint summary, and the summary the tree built. Each closed triangle gives one more oracle-consistent node one level up, and repeating the step from leaves to root proves the proposition. C2 covers the one case the induction misses: a stored summary summarized again after the tree is built. The formal statement and the full induction are in Appendix B.3.

The proposition is an accounting result, and that is its value. The root claim names one unobservable comparison: whether a single string, the root summary, carries the same task judgment as an entire document nobody is going to read in full. That comparison cannot be checked directly. Checking it would mean reading the document in full, the very cost the tree exists to avoid. The proposition converts it into a finite population of local claims, each naming a node, an input, and a

check a sampled audit can run. Local claims can be sampled, audited, and supervised.

That conversion also says where supervision can attach. Labels, rewards, or preferences collected on a document are useful for training or evaluation only if they track something a smaller object can also carry. The condition is simple: whatever assigns the label to the document must depend on the document only through its oracle value, $f^*(x)$, and not on any other detail of the raw text. When that condition holds and the laws hold on the realized tree, the root summary can receive the same label the document would have received, because $s_{\text{root}} \sim x$ makes the two indistinguishable at the oracle:

$$\text{supervision on } x \text{ depends only on } f^*(x) \implies \text{supervision on } x \text{ transfers to } s_{\text{root}}.$$

Labels, rewards, and preferences can then move from raw documents to tree summaries. The same argument runs one level down. When the oracle itself decomposes over spans, so that a document-level value is built out of span-level values, local labels on those spans supervise exactly the pieces the merge triangle carries to the root. They are direct supervision, not noisy proxies for a document-level target. Preference-learning objectives such as DPO and GRPO are two downstream signals that move under this condition; the condition is specific to neither.

Real summarizers rarely hit the laws exactly, so the useful version of the proposition is approximate. When the local checks hold only up to some distortion, the gap between the root summary's oracle value and the document's oracle value is bounded by a transport constant (fixed by the downstream use of the summary) times the audited local distortion accumulated on the realized tree. Section 4 builds the estimator for that distortion, states the bound on top of it, and shows the bound holding on realized trees.

4 The Sampled Node Audit

The guarantee of Section 3 is conditional: the laws must hold on the tree that was actually run. The audit measures whether they did. It is a survey of the deployed tree. Every leaf, merge, and recompression call in a realized tree is a place where oracle-consistency could have failed, and each is a fixed, finite object once the tree is built. Realized nodes form a finite population. Sampling them with logged inclusion probabilities gives design-based estimates of how often, and by how much, the laws failed. Each estimate comes with a confidence envelope. The logic is the survey statistician's: learn about a finite population from a drawn sample. Here the population is tree nodes.

The population. Fix a deployed tree over document x. Let $U(x)$ be the finite population of realized nodes: every leaf, every merge, every recompression call the tree actually made. $U(x)$ is the exact set of calls the tree made to reach its root summary. There is no superpopulation. Each node $u \in U(x)$ carries a logged inclusion probability $\pi_u \in (0, 1]$, the chance the audit design assigns to including u in the sample. The auditor samples a subset of $U(x)$, queries the oracle or an accessible judge on the sampled nodes, and reports design-based estimates of the population's violation rate and mean distortion, each with a confidence envelope.

Audit units by law. Each law names its own audit unit, because each law is a claim about a different kind of node. Leaf sufficiency (C1) is audited at leaves: does the leaf summary carry the oracle value of its raw span? Idempotence (C2) is audited at recompression calls: does resummarizing a stored summary change its oracle value? Merge consistency (C3) is audited at merge nodes: does the merged summary carry the oracle value of the concatenated span below it? The audit samples these unit types, not documents. A single sampled leaf or merge node answers one law's question

without touching the rest of the tree. The three questions are three separate finite-population estimation problems sharing one estimator. The merge query comes in two forms: against the raw children where they still fit inside the judge's context window, and against the joint summary deeper in the tree, where the children are already stored summaries. The audit design fixes which form runs at each depth (Appendix C).

Two outputs. Querying a sampled node produces two numbers. The *violation rate*, the share of sampled units where the oracle value moved by more than a tolerance, diagnoses which law fails and how often. A low C1 rate with a high C3 rate says the leaves are fine and the merges are not. The *mean distortion*, the average size of the oracle-value gap across sampled units, feeds the accuracy bound below. Both come from the same sample: the rate is the diagnostic, the mean is the input to the guarantee. For the leaf stratum the violation rate is the graded share of a uniform sample of n_1 leaves,

$$\hat{p}_{\mathrm{C1}} \;=\; \frac{1}{n_1}\sum_{i=1}^{n_1}\mathbb{I}\{\, d_{\mathcal{Y}}(f^*(g(b_i)), f^*(b_i)) > \tau \,\}, \tag{1}$$

the share of sampled leaf summaries whose oracle value moved by more than the tolerance τ. The C2 and C3 rates are the same shape on their own strata; Appendix C states all three.

The estimator. The audit queries a sample. A census of $U(x)$ against the oracle is the cost the tree exists to avoid. The tool for turning a sample into a population estimate under known, possibly unequal inclusion probabilities is the Horvitz–Thompson estimator [Horvitz and Thompson, 1952]:

$$\hat{\mu} = \frac{1}{N}\sum_{i\in U(x)}\frac{Z_i}{\pi_i}Y_i. \tag{2}$$

In words: each sampled node's measured distortion Y_i (its oracle-value gap under the query above) is reweighted by the inverse of its logged inclusion probability π_i, so rare-but-sampled nodes count for the many similar nodes that were not drawn. N is the population size, and Z_i indicates whether node i entered the sample. $\hat{\mu}$ is design-unbiased for the population mean distortion under the usual positivity condition on π_i, and it feeds the bound below. When the design samples each stratum uniformly at random, π_i is constant and Equation 2 collapses to an ordinary sample mean. The audit then grades a random sample of the tree's work, exactly as Equation 1 already does for leaves. The general form matters once the design oversamples, for example merges near the root.

What an error is, and where it goes. A mis-summarization is a law violation. If a leaf summary drops or distorts evidence that moves f^*, that call violates C1; if a cleanup pass shifts a stored summary's oracle value, it violates C2; if a merge loses an interaction between its children's spans, it violates C3. An error anywhere in the tree is a violation at a specific realized node, so it lives in the population $U(x)$ the audit samples from.

One wrong node can break the invariant at every ancestor on its path to the root, so the exact guarantee of Section 3 no longer applies. The approximate version takes over. The root's gap is bounded by the accumulated distortion of the realized calls (Equation 3 below). The accumulation charges magnitudes: Y_i is a distance, never a signed quantity, so an overstatement at one node is never netted against an understatement at another. When errors do cancel at the root (in an additive oracle, a leaf error that pushes the rollup one way can offset one that pushes it the other), the truth is better than the bound. The bound spends no assumption on cancellation.

Errors cannot hide from the audit, and the reason is the design, not luck. Every realized node carries a logged inclusion probability $\pi_i > 0$, and $\hat{\mu}$ is design-unbiased for the mean distortion over all of $U(x)$, the unsampled nodes included. A systematic failure mode (a summarizer that reliably drops one kind of evidence) hits many nodes at once, so it raises the population mean in proportion to how often it occurs. The more systematic the error, the more visible it is to a random sample. What a finite sample cannot do is point to the specific unsampled node where an error sits. It does not need to: the bound charges the estimated total, and the envelope B_{est} prices the chance that the sample understates it.

Calibration separates the judge from the oracle. The audit queries an accessible judge in the oracle's place: a cheaper model, a rubric, or a research assistant standing in for the expert who defines f^*. Querying f^* at every sampled node is the cost problem the tree was built to solve. B_{cal} is the residual between that judge and the task oracle. It is fixed by the calibration set and the confidence level chosen for the audit. More sampled node labels shrink the audit's own sampling uncertainty and leave B_{cal} unchanged. A judge that is the oracle sets it to zero.

This is design-based supervised learning's logic, repeated one level down. At the document level, a model's imperfect labels stand in for expert labels across a corpus, and a random subsample of gold-standard documents corrects the difference so that downstream estimates stay valid [Egami et al., 2024]. The audit runs the same design at the node level: the corpus becomes the realized nodes of a deployed tree, the imperfect surrogate becomes the stored summary standing in for its raw span, and the gold subsample becomes the sampled nodes whose oracle values are collected. The estimator and the correction logic carry over unchanged; only the population is new. Two approximations are in play at once: the tree's summaries approximate the spans they replace, and the accessible judge approximates the oracle that defines correctness. One random sample of gold-standard labels checks both, because the calibration set that measures B_{cal} and the audit sample that measures $\hat{\mu}$ share the same design.

The bound. The audit turns $\hat{\mu}$ and B_{cal} into an accuracy statement about the deployed tree's root prediction:

$$\text{gap} \;\le\; C \cdot \hat{\mu} + B_{\text{cal}} + B_{\text{est}}. \tag{3}$$

In words: gap is the error between the root summary's oracle value and the full-document oracle value; C is the transport constant from the propagation guarantee in Section 3, fixed by the tree's structure and independent of the audit sample; $\hat{\mu}$ is the Horvitz–Thompson distortion estimate from Equation 2; B_{cal} is the calibration residual just defined; and B_{est} is the sampling envelope around $\hat{\mu}$, shrinking as the audit budget grows. Each term is priced in the same units as the root's oracle value, so the bound is a single number a deployed tree reports alongside its prediction. The bound is an error budget: summarization error lands in $C \cdot \hat{\mu}$, judge error lands in B_{cal}, and the audit's own sampling error lands in B_{est}. An error anywhere in the system has exactly one term to land in.

Two results validate this instrument, then use it. The census-validated audit runs on the coded-manifesto trees, where every node carries an exact induced oracle value from the CMP codes, so the audit itself can be checked against ground truth before it is trusted anywhere else. At audit budgets $n \in \{50, 200, 800\}$, drawing twenty independent samples per budget with logged inclusion probabilities and computing $\hat{\mu}$ from Equation 2, the estimate tracks the census. For the merge-consistency violation rate the audit reports 0.163 ± 0.053 at $n = 50$, 0.143 ± 0.030 at $n = 200$, and 0.139 ± 0.011 at $n = 800$, against a census truth of 0.140; the envelope covered the census value in every replication at every budget. The mean-distortion estimate tracks its truth the same way

(0.0049 ± 0.0004 against 0.0050 at $n = 800$), as do the per-dimension estimates (Table 14). A second arm scores the same sampled nodes with an LLM judge in place of the census, so B_{cal} is measured, not assumed: the judge's calibration residual is 0.008. The deployed audit then runs the validated instrument where the census is unavailable: on the headline document-label trees from Section 6, sampling $n = 200$ realized merge nodes, querying the same LLM judge, and importing the measured calibration residual. The judge estimates a mean merge distortion of $\hat{\mu} = 0.065$, and assembling Equation 3 for merge consistency on the flagship measurement gives

$$\text{gap} \;\le\; C\hat{\mu} + B_{\text{cal}} + B_{\text{est}} \;=\; 0.065 + 0.008 + 0.050 \;=\; 0.122$$

with $C = 1$. The census audit answers whether the instrument works where truth is free to check. The deployed audit reports what it says where the tree is actually deployed.

5 Neural Operators and Language Models

The local laws constrain a function, not an implementation. Whatever plays the leaf and merge roles must keep the raw span's oracle value in a form the next step can use, and any function class that can do it can carry the tree. Section 2 named the hand-designed instance: a classical mergeable sketch, whose merge preserves its query by algebra, so the audit has nothing left to check. The learned instances live in one class, neural operators, and the class delivers the two properties the tree needs. A single learned operator can be evaluated at any resolution of the text, so one summarizer serves paragraph leaves and page leaves alike. And the class contains the transformer, so the prompted language model of Section 6 and the network trained against node labels there are members of one family, covered by the same laws and the same audit.[1]

5.1 Operators on Text

A neural operator is a learned map between function spaces [Kovachki et al., 2023]. The tree's calls fit that description directly. A span enters as a function, its embedding sequence indexed by position; the summary leaves as another function in the same space, ready to be an input one level up. The architecture is a pointwise lift, a finite stack of layers, and a pointwise projection:

$$\mathcal{G}_\theta = Q \circ \sigma_T(W_{T-1} + K_{T-1} + b_{T-1}) \circ \cdots \circ \sigma_1(W_0 + K_0 + b_0) \circ P. \tag{4}$$

In words: P widens each position's representation, each layer combines a local linear map W_t with a kernel integral K_t that lets every position see every other position, and Q reads the result back out. The kernel integral is the part that matters for summarization. It is what carries an interaction between two clauses that sit far apart, the information a merge must not lose. A Fourier neural operator (FNO) computes the kernel integral in frequency space, which makes the global mixing cheap [Li et al., 2021]; that is the implementation trained against induced node labels in Section 6. The form in Equation (4) is a function-space interface, and the FNO is one implementation of its kernel layers. The guarantees of Sections 3 and 4 are stated on f and g and never mention the parameterization.

[1] Text is the running instantiation of a general mathematical fact: the objects are strings over an alphabet, an oracle, and an operator, so the same laws, guarantees, and audit apply to any data whose summaries merge. The supplement makes the generality exact: on classical sketch inputs the tree reduces to the classical mergeable sketch byte-for-byte (HyperLogLog registers come back byte-identical), and the same interface covers Count-Min, quantile, and Markov-count sketches.

5.2 Resolution Independence

The property that makes this the right class is discretization invariance [Kovachki et al., 2023, Section 2.3]: the operator's parameters live in function space, not on any particular grid, so one parameter-sharing operator can be evaluated on multiple discretizations of the same underlying object, and refining the discretization improves the approximation rather than changing the target. Resolution independence is the plain name.

A compression tree needs exactly this property, because a leaf partition is a discretization of the document and leaf size is its resolution. Cutting a manifesto into paragraph leaves or page leaves changes how finely the same text is sampled; it changes neither the document nor the judgment. Discretization invariance means one learned operator can be shared across short leaves, longer merge inputs, and recompression calls near the root, with no retraining and no separate model per level.

Two invariances meet here, and they divide the work. Discretization invariance is architectural: it makes a level-invariant summarizer expressible, one set of parameters for every resolution. The local laws are semantic: they demand the same oracle value whatever the cut, and the audit checks whether the demand held. The flat leaf-size sweep of Section 6, near-constant agreement from paragraph leaves to page leaves, is what the two invariances look like when both hold.

5.3 Transformers as Neural Operators

The class is wider than its Fourier implementation, and the width is the point. Following Kovachki et al. [2023, Proposition 6], single-head attention agrees with a discretized kernel layer, a transformer block is therefore a neural-operator layer, and a finite encoder stack instantiates the architecture of Equation (4). Attention is one way to compute the kernel integral; Fourier mixing is another. The inclusion is architecture-level: it says nothing about a tokenizer, a serving stack, or a particular checkpoint. It says the thing the framework needs. A transformer summarizer sits inside the same operator class as the trained FNO, so nothing about the laws, the guarantees, or the audit changes when the summarizer becomes a language model.

5.4 The Training Objective

The laws enter training as penalties. The objective's exact form carries the framework's central design choice, so we state it in full. Let $\widehat{f}$ be the learned readout, calibrated against the task oracle exactly as the accessible judge of Section 4 is, and let g be the learned summarizer. Training minimizes

$$J(\widehat{f}, g) \;=\; L_{\mathrm{cal}}(\widehat{f}, f^{*}) \;+\; L_{\mathrm{root}}(\widehat{f}, g) \;+\; \lambda_1\, \mathrm{C1}_{\widehat{f}}(g) \;+\; \lambda_2\, \mathrm{C2}_{\widehat{f}}(g) \;+\; \lambda_3\, \mathrm{C3}_{\widehat{f}}(g). \tag{5}$$

Term by term: L_{cal} aligns the readout with the task oracle on a calibration sample, the first of the two alignments in Section 4. L_{root} fits the root prediction to the task labels. Each law term charges the average violation of its law over the realized calls of the training trees, measured by how far the call moved the value $\widehat{f}$ assigns: $\mathrm{C1}_{\widehat{f}}(g)$ on leaf calls, $\mathrm{C2}_{\widehat{f}}(g)$ on recompression calls, $\mathrm{C3}_{\widehat{f}}(g)$ on merge calls.

Nothing in Equation (5) constrains the space of learnable functions. The summarizer ranges over the entire operator class; no architecture choice, mask, or parameterization enforces a law. A violation is priced, never forbidden: the optimizer may trade a law violation against task fit wherever the trade lowers the objective, and whatever violations survive training are properties of the trained operator that the audit can find.

The penalties still point somewhere definite. An operator incurs zero law penalty exactly when it satisfies the three laws (an equivalence the supplement records as a theorem), so the weights

$\lambda_1, \lambda_2, \lambda_3$ act as a projection toward the subspace where the guarantees of Section 3 apply: small weights trade law violations against task fit, and large weights approach a hard projection. The dial has a useful endpoint. On synthetic oracles, where every span's oracle value is computable by construction, laws-only training drives the audited violations to literal numerical zero. The supplement gives the full result.

Training does not certify where the operator landed; the audit does. Equation (5) pushes g toward the law-satisfying subspace, and the sampled audit of Section 4 measures the distance that remains, law by law, on the tree actually deployed.

5.5 The Alternating Fit

The objective couples two learners, and we minimize it by alternating between them. The readout is trained first. With g held at its initialization, stage one fits $\widehat{f}$ against the calibration and root terms of Equation (5). Stage two holds $\widehat{f}$ fixed and trains g against it, root fit and law penalties together. Further rounds repeat the pair, each side refit against the other's current state. We call the model after stage one the *f-stage* and the model after stage two the *g-stage*; the results tables report both.

The readout goes first because every law penalty is measured through it. Each term of Equation (5) charges how far a call moved the value $\widehat{f}$ assigns, so until the readout tracks the oracle, a violation has no price. The summarizer's initialization makes the first stage meaningful: the untrained merge starts at the mean of its children (a warm start toward the additive rollup), so the f-stage is a full tree whose composition is generic averaging. Its accuracy is what leaf reading alone achieves.

The f-stage is therefore a baseline as well as a warm start. The increment from f-stage to g-stage, holding data, splits, and readout training fixed, is the value of learned composition and nothing else. The prompted families run the same schedule with selection in place of gradients: the scorer prompt is optimized against the current summaries, then the summarizer prompt against the current scorer. Section 6 reports both stages across leaf sizes, and the supplement reports the prompt ladder in full.

5.6 Prompted Language Models

The widest-deployed members of the class need no gradient training at all. A language model under a fixed prompt is a text-to-text operator: it takes a span, or a pair of summaries, and returns text in the same space. Prompting or fine-tuning selects which operator in the family is in play, exactly as gradient training selects an operator in an FNO family, and the transformer inclusion above says the selection happens inside the class everything here already covers. The laws apply unchanged: leaf sufficiency, idempotence, and merge consistency name the same three checks whatever produced the summary. The audit is stated on inputs and outputs. It samples calls and scores what came back, so a model served behind an API is as auditable as a model run on local hardware, with no gradient access, weight inspection, or training-time hook. The audit built for a trained network carries over to a prompted one unchanged.

Section 6 runs exactly this: a prompt-only tree over one open-weight language model, judged against a real measurement target.

6 Measuring Party Positions from Manifestos

Political scientists want a handful of numbers out of every party manifesto: where the party sits on the left-right axis, how much it emphasizes the environment, how much autonomy it grants

subnational government. The Manifesto Project publishes exactly these judgments for thousands of platforms, at two levels of resolution: one layer scores whole platforms, the other scores individual clauses inside them. Section 6.1 describes the corpus and both label layers.

Manifestos are a long-document measurement problem before any labels enter the picture. A typical platform on the benchmark roster runs 168K characters, well past what a single prompt call can hold at a working context budget of $c = 4$K characters. Scoring one requires either truncation, which drops evidence, or compression, which is exactly the operation the local laws exist to check. The oracle here is not a closed-form statistic. It is the rubric a trained coder or a careful reader applies: for a span of text x and a policy dimension d, $f_d^*(x)$ is the seven-point score that rubric would assign. There is no hand-designed mergeable sketch for a judgment like this. It has to be read.

Section 6.2 shows a tree of local LLM calls matching the same model's full-document read on the platform-level scores. Section 6.3 checks that the match holds regardless of how the document is cut into leaves. Section 6.4 moves inside the tree: it validates the clause-level labels against the platform-level scores, then trains a summarizer on them directly. Section 6.5 splits that trained substrate by policy dimension and locates the ceiling the label system sets on the external target. Section 6.6 draws the general lessons.

6.1 The Manifesto Project Data

The Manifesto Project (MARPOR) codes and publishes the text of party election platforms from parliamentary democracies around the world [Lehmann et al., 2025b]. Its central release, the Manifesto Project Dataset (MPDS), gives one row per platform: the party, the election, and a set of summary scores computed from the coding underneath. The release used here has 5,285 platform rows spanning 67 countries and elections from 1920 to 2025.

A subset of these platforms also carries the Manifesto Corpus, the Project's release of the underlying text broken into quasi-sentences and hand-coded clause by clause [Lehmann et al., 2025a, Merz et al., 2016]. A quasi-sentence is a clause-level fragment of text, typically shorter than a full sentence: coders split each sentence into as many quasi-sentences as it contains distinct policy statements and assign each one a single code from a 56-category scheme. 2,157 of the MPDS platforms overlap this coded release, contributing about 2.27 million coded quasi-sentences across 42 languages. The document scores exist for every platform; the clause-level codes exist only for this later, digitized subset.

Because the codes are counts, two useful scores fall out by simple arithmetic. RILE, the standard left-right index, is the normalized difference between the count of quasi-sentences in one of 13 designated right-coded categories and the count in one of 13 left-coded categories [Laver and Budge, 1992]. Seven further domain shares sum the same counts within a policy domain: external relations, freedom and democracy, political system, economy, welfare and quality of life, fabric of society, and social groups. Both constructions are additive. The score for a document is the sum of the scores for its parts, computed over whatever span of text is convenient.

None of this is the target researchers actually want from a platform: a holistic judgment of where a party sits on a policy dimension. Benoit et al. supply exactly that, an expert-survey benchmark of 235 party platforms, each scored on six seven-point policy scales (economic, social, immigration, European integration, environment, decentralization) by the harmonized mean of several trained experts [Benoit et al., 2025]. Human experts scoring the same platform independently agree with each other at a macro Pearson $r = 0.873$; this split-expert agreement is the reference any automated method is judged against.

The two layers do different jobs. The expert-survey scores exist only at the document level. No

expert ever rated a paragraph or a merge, so they check a root prediction and nothing beneath it. Call this the *external target*. The quasi-sentence codes exist at the clause level and sum over any contiguous span, so they induce a value at every leaf, every internal merge, and the root of a tree built over the same document, checkable at the root against the Project's own published document score. Call this the *internal rollup*. Sections 6.2 and 6.3 use the external target; Section 6.4 validates and trains against the internal rollup.

6.2 The Document-Label Result

A prompt-only C-Tree over one open-weight model reads party manifestos as accurately as the same model reading the whole document at once. Once the root score matches the uncompressed read, everything else the tree provides (bounded local calls, auditable nodes, a leaf partition the answer does not depend on) comes at no accuracy price.

The setup: the summarizer g and the readout f are both a single open-weight checkpoint (Gemma-4-31B-NVFP4) under fixed rubric prompts, with no fine-tuning and no architecture beyond the prompts themselves. A manifesto is partitioned into contiguous character spans of size c; each leaf span is summarized under a rubric naming the preservation categories drawn from the expert-survey scoring context; sibling summaries are merged pairwise up a binary tree under the same rubric; and the root summary is scored on each of the six policy scales of Section 6.1. The comparison set is Benoit et al.'s 235-platform benchmark, correlated dimension by dimension and averaged into a macro Pearson r.

The primary comparison isolates the cost of compression: the C-Tree against the same model reading the full document directly, on the full matched roster. The flat read at a 48K-character truncation reaches macro $r = 0.771$; leaf summaries concatenated without a learned merge reach 0.800; the C-Tree reaches 0.787 at an 8K-character leaf and 0.783 at 4K, after the document has been cut into much smaller local units and rebuilt through merges. Breaking the document apart and reassembling it through local calls costs nothing against the same model's uncompressed read.

The external references locate this band among published work the tree never saw. Benoit et al.'s single-model replications span macro $r = 0.752$–0.793 (Gemma-3-27B at 0.793); the tree's 0.787 sits inside that band with every call bounded by an 8K-character window. The proprietary three-model, eighteen-score ensemble reports 0.817, and split-expert agreement sits at 0.873. The per-dimension pattern tracks the published baselines on five of the six scales (Figure 2). Decentralization is the standing exception, taken up in Section 6.3.

Table 3 carries the comparison, and its baseline rows price each stage of the pipeline. The scorer-only row (macro $r = 0.743$) runs the scoring stage alone on Benoit et al.'s released summaries: what the scorer contributes with no tree at all. The flat read (0.771) is the signal available to one direct pass over truncated raw text. The concat row (0.800) shows the signal surviving the first local compression step, before any merge runs. The C-Tree rows show it surviving the full local architecture, merges included. The concat row has no bounded window: its scorer reads every leaf summary at once, an input that grows with the document, while every tree call stays inside one fixed window. The jump from the scorer-only value to the 0.77–0.80 band is the summarization stage's contribution.

The published ensemble makes 18 scored calls per manifesto-dimension pair across three proprietary frontier models. A per-dimension C-Tree makes one summarize call per leaf, one merge call per internal node, and one score call, all to one open-weight model on local hardware: about 45 calls per manifesto-dimension pair at $c = 8$K on this roster, whose manifestos average 168K characters (Appendix D reports the measured accounting). A joint tree sharing summarize and merge calls across the six dimensions makes ≈ 8 per pair. The published measurement can be reproduced, and

audited, with no proprietary model in the loop.

Method	Macro r	Calls/manifesto-dim.	Model class
Split-expert reference	0.873	—	human coders
Proprietary ensemble (18 scores)	0.817	18	3 frontier LLMs
Gemma-3-27B-IT replication	0.793	1	open-weight
Scorer-only (Benoit summaries, no tree)	0.743	1	open-weight
Full-document read, 48K-char. truncation	0.771	1	open-weight
Leaf summaries concatenated, no merge, $c = 8$K	0.800	≈40	open-weight
C-Tree, $c = 8$K leaves	0.787	≈45	open-weight
C-Tree, $c = 4$K leaves	0.783	≈90	open-weight

Table 3: Abridged headline comparison. Macro Pearson r against Benoit et al. expert-survey means, averaged over six policy dimensions. The five lower rows share one campaign: the full matched roster, one open-weight model (Gemma-4-31B-NVFP4), one decoding configuration; only the reading strategy varies. Calls/manifesto-dim. counts language-model calls per manifesto-dimension pair, measured from run artifacts on this roster; a joint tree that shares summarize and merge calls across the six dimensions makes ≈ 8 per pair (Appendix D). The scorer-only row runs only the scoring stage on Benoit et al.'s released summaries. Full per-dimension table, per-cell ablations, and the compute accounting appear in the appendix.

6.3 Leaf-Size Invariance

The root score does not depend on how the document is cut into leaves. Leaf size is a nuisance parameter of this instrument, not part of what is being measured, and the leaf-size sweep checks that claim (Figure 3). Across a sixteen-fold range of leaf sizes ($c = 4$K to $c = 64$K characters), macro r moves by 0.027. The sweep is evidence that the local laws hold well enough on this pipeline to make the estimand indifferent to the leaf partition; the audit of Section 4 checks the laws directly.

Decentralization is the instructive departure from the pattern. Its correlation improves as leaves shrink, from 0.489 at $c = 64$K to 0.584 at $c = 2$K. This is the one dimension Benoit et al. themselves flag as a boundary case, where coders and experts diverge on what evidence resolves a party's position; their own split-expert reference on it is 0.78, well below the ≥ 0.88 references for the other five dimensions. Finer localization helps exactly here, because the decisive clause is less likely to be diluted inside a large leaf.

The result holds where the long-document problem is hardest. Among manifestos longer than 100K characters, the per-dimension C-Tree reaches Pearson $r = 0.83$–0.90 on the five non-decentralization dimensions, with decentralization at ≈0.40 matching its corpus-wide pattern (Table 8). The longest documents are inside the headline result, not exceptions to it. Splitting by election era, macro r runs 0.702 over the four dimensions scored in 1989–1999 (immigration and European integration have no cases in that window), 0.759 in the 2000s, and 0.806 in the 2010s, with decentralization the weakest dimension in every era (0.36–0.48). No single era carries the result. Appendix D reports the full protocol, the per-dimension and per-leaf-size grids, the scorer-only and exact-model baselines, decoding-sensitivity checks, and the call-count accounting.

The prompted family also runs the alternating fit of Section 5.5: the scorer optimized against the current summaries, then the summarizer against the current scorer. The resulting ladder reproduces the same leaf-size invariance under iterative training, and the supplement reports it in full.

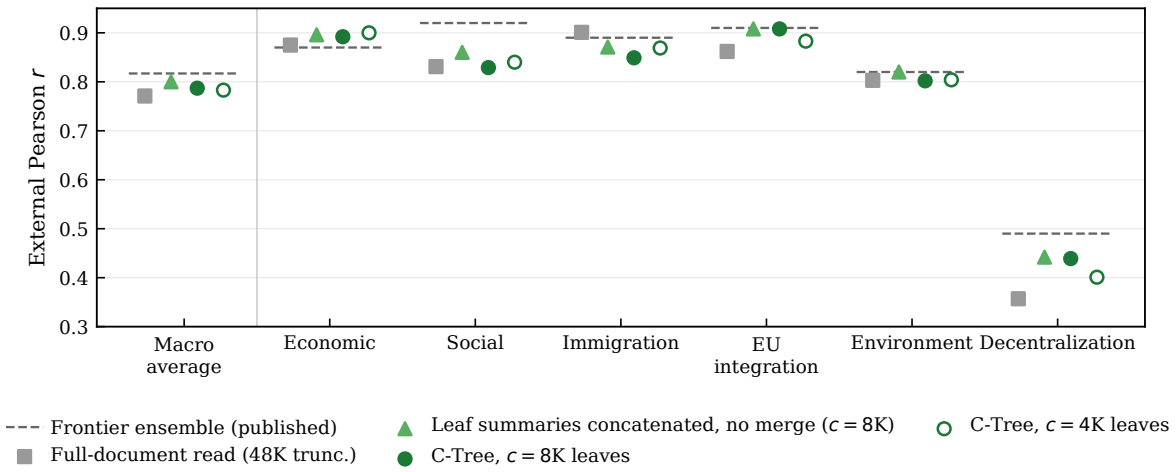


Figure 2: Per-dimension parity across reading strategies. Each column is one policy dimension (macro average at left); markers are the four reading strategies of Table 3, one campaign on the full matched roster; the dashed tick is the published proprietary ensemble. On five of six dimensions the four strategies and the ensemble land in one band: the choice of reading strategy, including whether the document is decomposed at all, does not move the measurement. Decentralization drops for every strategy alike, so its difficulty belongs to the dimension, not to the tree.

6.4 Node Labels and the Sentence Ladder

The second supervision layer moves inside the tree, and it starts with a check on the labels themselves. Rolling the quasi-sentence codes up to the document level and correlating against the Manifesto Project's own published document scores, across 2,157 coded platforms, gives Pearson $r = 0.9975$ for RILE and $r \in [0.924, 0.999]$ across the seven domain shares (Table 10). The published document label is, to numerical precision, the additive rollup of the induced node labels. The label system checks out before anything is trained on it: a method that recovers the right per-node states and merges them lawfully will recover the published document score.

A learned merge trained against these targets composes. A Fourier neural operator summarizer and readout is fit to the induced node targets under the alternating schedule of Section 5.5, on a roster of 217 Western European platforms (train 139 / validation 30 / test 48). Leaves are ℓ consecutive sentences, $\ell \in \{1, 2, 4, 8, 16\}$; the sentence is the smallest readable span, and each sentence's label aggregates its quasi-sentences' codes. One platform is excluded because its text lacks terminal punctuation, so the deterministic sentence grouping degenerates on it. Table 4 reports the f-stage and g-stage external Pearson against the published RILE index on the held-out test split.

The merge composes at every leaf size: the g-stage beats the f-stage in every row, with the largest lifts at the smallest leaves, where merge chains are deepest and generic averaging loses the most. The improvement peaks at an intermediate leaf, $\ell = 2$ sentences, where external r reaches 0.713. The same shape held on the retired quasi-sentence axis, at the same height, with the peak again at neither the smallest nor the largest leaf tried. Composition has to be learned even with node labels available, and it is learned best where a leaf carries a moderate amount of labeled content.

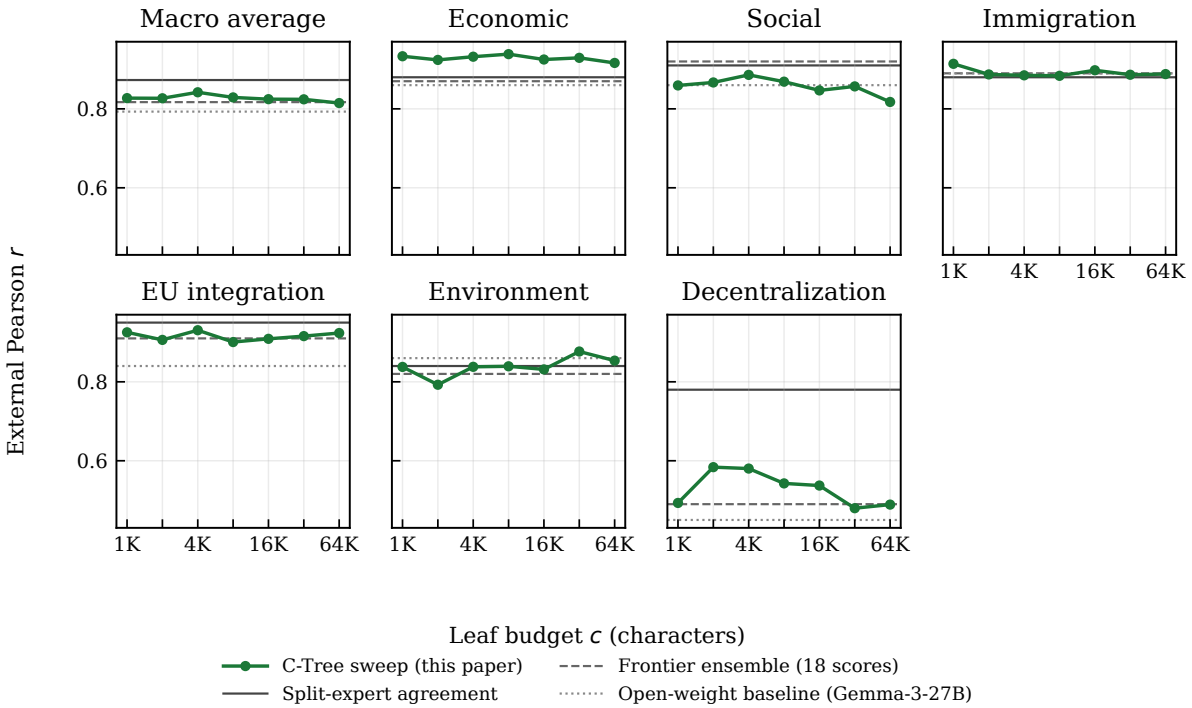


Figure 3: Leaf-size invariance by policy dimension. Each panel plots external Pearson r against expert-survey means as the leaf budget varies from $c = 1$K to $c = 64$K characters (log axis); the top-left panel is the macro average. Horizontal references mark split-expert agreement (solid), the published frontier ensemble (dashed), and the best open-weight single-model baseline (dotted). The finding is the flat slope: over the $c = 4$K–64K range the macro average moves by 0.027, so the measurement does not depend on how the document is cut. Decentralization (bottom row) is the exception discussed in the text. Sweep cells run on the frozen 40–50-manifesto sweep roster; full-roster values at matched cells sit ≈ 0.03 lower (Table 3), so the invariance claim concerns the spread across leaf sizes, not the absolute level.

6.5 Weaker Substrates and the Construct Ceiling

Everything above ran a 31-billion-parameter model. The node-label ladder needs none of that scale: its substrate is a 300-million-parameter embedding model (embeddinggemma-300m) that encodes each leaf once, and a small Fourier neural operator trained against the induced node labels alone, with no expert scores in the loop. This subsection asks how far the small substrate goes: whether composition survives a split by policy dimension, and where it stands against the external target it was never trained to hit.

Splitting the sentence-leaf bundle at $\ell = 8$ into its eight compact dimensions shows where composition succeeds (Table 12). Two domains compose well and stably across three training seeds: economy reaches external $r = 0.669 \pm 0.010$ and welfare and quality of life reaches 0.616 ± 0.003. Fabric of society composes on one seed and collapses on the others (0.419 ± 0.253). The remaining four domains (external relations, freedom and democracy, political system, social groups) collapse to a mean near zero with a standard deviation of comparable size: the trained readout predicts a constant, and the merge inherits the collapse rather than causing it. The split tracks label density. The two composing domains carry a mean coded share of 0.25–0.32 per leaf with 42–51% of leaves at zero; the four collapsing domains carry 0.07–0.11 with 72–86% of leaves at zero. Density, not the

Leaf (ℓ sentences)	f-stage ext. r	g-stage ext. r	g-stage MAE
1	0.459	0.579	0.057
2	0.582	**0.713**	0.051
4	0.620	0.699	0.053
8	0.625	0.647	0.061
16	0.631	0.664	0.061

Table 4: Neural-operator (FNO) summarizer trained on the induced node targets, scored against the published RILE index. External Pearson r, test split ($n = 48$), single training seed. The g-stage merge is trained against the f-stage readout under the alternating fit of Section 5.5. Leaves are ℓ sentences on a roster pinned at $n = 217$, one platform excluded for degenerate, unpunctuated sentences. Full grid and internal metrics appear in the appendix; the LLM-family counterpart appears in the supplement.

dimension's identity, predicts whether composition works, and the failure is legible when it comes.

The rollup check of Section 6.4 shows the internal target is trustworthy. It says nothing about how much of the external target the codes determine. Regressing the eight compact-state features against the Benoit expert means by ordinary least squares gives five-fold cross-validated R^2 between 0.035 (decentralization) and 0.472 (immigration) (Table 10). The expert construct is a holistic judgment the additive category counts only partly determine. Call this the *construct ceiling*: it bounds what any method trained on the quasi-sentence codes alone can reach against the external target, and it is set by the label system, not by the model. The ceiling is defined on the six expert dimensions; the dense/sparse split above is a separate mechanism, driven by label density, and the two orderings should not be read as one ranking.

The construct ceiling reappears when the trained substrate is scored against the external target directly. Training the neural-operator family at $\ell = 16$ and scoring roots against expert means (Table 13) gives $r = 0.734 \pm 0.016$ on economic (also the dimension the recipe was tuned on; Appendix E details the caveat), 0.632 ± 0.034 on environment, and 0.527 ± 0.064 on social (mean $\pm$ sd over three seeds), with decentralization (0.271 ± 0.009) and European integration (0.169 ± 0.010) at the floor. The match to the ceiling is exact at the extremes: the two lowest-ceiling dimensions land at the floor of the scored table, and the middle of the ordering is looser (Appendix E discusses immigration's wide seed spread). The top of the table recovers most of the signal a full-document frontier-LLM read reaches on the same task ($\approx$0.86) at a small fraction of the per-call cost; the floor recovers almost none of it.

6.6 Lessons

The two supervision levels answer two different measurement questions, and neither substitutes for the other. Document labels (Section 6.2) check whether a tree of local calls preserves an external judgment it was never trained against; the leaf-size sweep (Section 6.3) answers that question directly, and it holds because the summarizer is already competent, not because it was optimized against the sweep. Node labels (Sections 6.4 and 6.5) check whether a summarizer can be trained to compose lawfully from local supervision alone; the sentence-leaf ladder and the dense/sparse split answer that question, and the answer depends on how much signal a dimension carries in the induced labels.

The construct ceiling is a general lesson, not a manifesto-specific one. However well a local-law objective composes internally, it is bounded by what the local labels determine about the target a

researcher actually wants. Checking the internal rollup before training (here, $r = 0.9975$) separates a label-system failure from a model failure. Checking against the external target after training separates a construct that the codes reach from one they do not.

The node census that validates rollups here is the same object the sampled audit of Section 4 approximates under a budget. Every node of a coded manifesto carries an exact induced oracle value because the corpus happens to supply exhaustive local labels. Most deployed corpora do not. Where node labels are exhaustive, as here, checking oracle-consistency at every node is a census. Where they are not, sampling a subset of realized nodes with logged inclusion probabilities and comparing against exact values on that subset is the same check run on a budget. The manifesto corpus is the one place where the audit's target quantity (the population of realized nodes and its true violation rate) can be computed directly, and it validates the estimator the audit relies on everywhere else.

7 Discussion

The guarantees of Section 3 preserve the oracle when the local laws hold, and the audit of Section 4 checks the laws on the deployed tree. C-TreePO therefore applies when the task admits a locally checkable sufficient state: an oracle whose value on a large span can be recovered from oracle-consistent summaries of its parts. When a law fails, the audit shows the failure as a nonzero violation rate on the affected law, with a confidence envelope around the estimate. Rhetorical tone and discourse-level meaning are the standing counterexamples. Replacing a span with an equivalent one changes how neighboring text reads, so the oracle does not decompose. On such a task the audit reports high C1 or C3 violation rates, and the failure is visible, not silent.

Portability. A new domain needs four things to run the same recipe. A task oracle: a rubric, a coding scheme, or expert judgment that assigns a value to spans of the input. An input that splits into contiguous spans: pages, sections, scenes, clauses, whatever unit the oracle can be evaluated on. A judge for sampled audits: an accessible stand-in for the oracle, cheap enough to query on a sample of realized nodes. Local labels, optionally, for training the leaf and merge operators directly. The audit checks the operators after the fact either way. The manifesto study in Section 6 is one instance of this class, with the CMP rubric as the oracle, manifesto sections as spans, an LLM judge for the sampled audit, and quasi-sentence codes as local labels. Comparative-politics scoring is one point in a broader family that includes clinical narrative review, legal-corpus analysis, and policy-agenda coding. The recipe fits anywhere a rubric-driven judgment is read off a document too long to score directly every time.

Related methods. Long context solves input-size limits: a large enough window makes truncation rare. Oracle preservation is a separate property, and it is the one measurement needs. A single read through a wide window carries no record of what evidence the model tracked or discarded. Nothing about the window size says whether the model's rubric decision matches an expert's. Retrieval-augmented methods solve relevance filtering: they surface the passages most likely to bear on a query, saving the model from reading everything. A target that combines evidence across passages the retriever never surfaced is out of reach there. Nothing in the pipeline records what was dropped. Surrogate-label methods solve annotation cost: an imperfect model label stands in for an expensive expert label, and design-based correction recovers valid downstream inference from the imperfect signal [Egami et al., 2024]. The node audit in Section 4 applies the same design-based logic to nodes of a tree, in place of documents of a corpus. Mergeable sketches solve exact aggregation

for a fixed query class: a distinct-count sketch merges losslessly because its merge rule is proved correct by algebra [Agarwal et al., 2013]. A learned summarizer has no such proof, so the sampled audit checks its merges in place of the proof.

Measurement interpretation. The calibration residual B_{cal} is part of the instrument, not overhead on top of it. Every applied measurement that stands an accessible judge in for an unreachable target carries this residual, whether the judge is a research assistant, a coding scheme, or a language model. The audit prices the residual. Reporting B_{cal} alongside the root prediction turns the tree into an instrument with a documented calibration, the same way a survey instrument reports its measurement error alongside its point estimate.

Honesty and learned partitions. The trees studied here fix the leaf partition in advance, and the local laws make the measurement invariant to that choice. Section 6 shows the root estimate stable across a sixteen-fold range of leaf sizes. Causal forests offer the next step. Honest sample splitting uses one part of the data to choose a partition and a disjoint part to estimate on it. That is what lets a data-driven partition still support valid inference [Athey and Imbens, 2016, Wager and Athey, 2018]. Applied to a compression tree, leaf boundaries and leaf sizes become learned quantities: split where the oracle value changes and grow leaves where it is flat, using one held-out fold to locate the partition. The audit stays valid under this design because the nodes it samples were located on data disjoint from the fold used to fit the partition. The audit's inclusion probabilities and the resulting Horvitz–Thompson estimate then carry the same guarantees they carry under a fixed partition. This turns the leaf partition from a design choice fixed before the tree ever sees data into an adaptive object the tree learns, honestly, from a split of its own.

Preference optimization. Section 3's moving-supervision condition has one direct use for preference learning. A preference-optimization objective collected on full documents, such as DPO or GRPO, transfers to tree summaries when the preference signal factors through the preserved oracle value. That is, the annotator's choice between two documents depends on the documents only through f^*, not on surface text the tree was free to discard. The same condition licenses collecting preferences at the node level when the oracle decomposes over spans, because local preferences on the composing pieces then target the same population object as a preference collected on the whole. Readability and stylistic preferences sit outside f^*. A richer oracle or a separate alignment channel handles those.

8 Conclusion

Locally oracle-consistent summaries compose. If every leaf summary is oracle-consistent with its raw leaf and every merge preserves that consistency across the concatenation of its children's spans, the triangle argument of Section 3 carries the invariant from leaves to root by induction: the root summary preserves the task oracle on the full document.

The condition is auditable, and that is what makes it usable. A deployed tree turns the invariant into a finite population of realized nodes. Sampling that population with logged inclusion probabilities and gold-standard oracle values on the sample gives a design-based estimate of where and how often each law fails, in place of a proof taken on faith.

On a real measurement target, the invariant held at parity. A prompt-only C-Tree over one open-weight model, with no fine-tuning and no architecture beyond the prompts themselves, matched the same model's full-document read on manifesto scoring (macro Pearson 0.787 versus 0.771), and

performance was nearly constant across leaf sizes, moving by 0.027 as leaves grew from about a paragraph to tens of pages. Quasi-sentence node labels located the mechanism: learned merges compose where the coded signal is dense and fail exactly where the labels run thin.

C-TreePO is valuable when a task admits a locally checkable sufficient state: a leaf summary that keeps what the oracle needs and a merge that keeps what crosses the boundary. It is honest when no such state exists: the audit reports the failure as a nonzero violation rate. The tree is useful because it creates auditable local claims.

Overview of the Appendices

Five appendices support the main text. Appendix A collects the notation in one table. Appendix B proves the propagation guarantee of Section 3, so every step from the three laws to the root bound can be checked. Appendix C gives the estimation machinery behind the audit of Section 4: the per-law estimators, how their guarantees scale across a tree, and the honesty protocol for data-built trees. Appendix D reports the document-label replication behind Sections 6.2 and 6.3: the corpus and protocol, the full comparison grid, the robustness slices, and the call-count accounting. Appendix E reports the node-label experiments behind Sections 6.4 and 6.5: the label construction, the rollup validation and construct ceiling, the sentence-leaf ladder, and the census-validated audit. A separate supplement holds the full technical record (the complete result ladder, preference-optimization transport, neural-operator theory, numeric example suites, sketch lineage, training dynamics, the machine-checked proof artifacts, the software package, and estimation variants); the text cites it where used.

A Notation

Appendix A collects the notation in one table, grouped by role: spaces, functions, tree objects, audit statistics, and relations.

B Proofs

Appendix B proves the propagation guarantee: the three laws restated as labeled equations, the merge-triangle lemma, then the induction that carries oracle-consistency to the root.

B.1 Preliminaries

Section 2 states the three local laws in prose and in card form (Table 2). The proofs below cite them as displayed, labeled equations, so they are restated here once with those labels.

Condition 1: Leaf sufficiency (C1).

$$\forall \text{ realized leaf } b: \quad g(b) \sim b. \tag{C1}$$

Table 2's first row states this in card form: the leaf summary must keep the evidence the task needs.

Condition 2: Idempotence (C2).

$$\forall \text{ summary } s \in \text{range}(g): \quad g(s) \sim s. \tag{C2}$$

Table 2's third row states this in card form: resummarizing an already-stored summary must not change its oracle value.

Symbol	Meaning
Spaces and Structures	
$\mathcal{X}$	Set of all finite token sequences (free monoid)
$\oplus$	Concatenation operator on $\mathcal{X}$
$\mathcal{Y}$	Oracle output space (equipped with metric $d_{\mathcal{Y}}$)
T	Full binary merge tree over leaf blocks
Functions	
$f^* : \mathcal{X} \to \mathcal{Y}$	Oracle: maps any document to the task value a researcher ultimately wants
$g : \mathcal{X} \rightsquigarrow \mathcal{X}$	Summarizer (possibly stochastic); the learned or hand-designed compressor at every node
$f : \mathcal{X} \rightsquigarrow \mathcal{Y}$	Readout: turns the root summary into the reported task value
$d_{\mathcal{Y}} : \mathcal{Y} \times \mathcal{Y} \to [0, \infty)$	Metric on the oracle output space
Tree Objects	
b_i	Realized leaf block (contiguous substring of the document x)
$S(u)$	Latent raw span under node u (concatenation of descendant blocks)
s_u	Stored summary at node u
u_L, u_R	Left and right children of node u
r	Root node
Audit Statistics	
$U(x)$	Finite population of realized nodes (leaves, merges, recompression calls) for document x
N	Population size, $N = \lvert U(x) \rvert$
π_i	Logged inclusion probability for node i under the audit sampling design
Z_i	Inclusion indicator for node i ($Z_i \in \{0, 1\}$)
Y_i	Measured distortion (oracle-value gap) for sampled node i
$\hat{\mu}$	Horvitz–Thompson estimate of mean population distortion (Eq. 2)
B_{cal}	Calibration envelope: residual between the accessible judge and the oracle
B_{est}	Estimation envelope: sampling-uncertainty around $\hat{\mu}$
C	Transport constant from distortion units to root-prediction objective units
Relations	
$u \sim v$	Oracle consistency: $d_{\mathcal{Y}}(f^*(u), f^*(v)) = 0$

Table 5: Notation dictionary for the symbols used in Sections 2–4. Manifesto-specific symbols and the supplement's neural-operator and preference-learning machinery are defined where they are used.

Condition 3: Merge consistency (C3).

$$\forall u, v \in \mathcal{X}: \quad \underbrace{u \oplus v}_{\text{raw parent}} \sim \underbrace{g(u \oplus v)}_{\text{joint summary}} \sim \underbrace{g\big(g(u) \oplus g(v)\big)}_{\text{tree's route}}. \tag{C3}$$

Table 2's second row states this in card form. The merge triangle of Section 2 names its three vertices: the raw parent, the joint summary from one call on the whole span, and the summary the tree's own two-children-then-merge route produces.

Each law is the same demand (a realized call of g returns something oracle-consistent with what it received) applied to a different call: C1 to a leaf call, C3 to a merge call, C2 to a recompression call.

B.2 The Merge Triangle

The induction that carries oracle-consistency from the leaves to the root repeats one step: given two already-consistent child summaries, the merge that combines them is itself consistent with the raw parent span. The following lemma isolates that step.

Lemma 1 (Merge triangle)**.** *Fix raw child spans $u, v \in \mathcal{X}$ and realized child summaries $z_u, z_v \in \mathcal{X}$. Suppose: (i) $z_u \sim u$ and $z_v \sim v$ (the children are already oracle-consistent); (ii) Assumption 1 (context compatibility); (iii) $g(u \oplus v) \sim u \oplus v$ and $g(z_u \oplus z_v) \sim z_u \oplus z_v$ (the first link of C3, instantiated at both the raw-span call and the realized child-merge call). Then, almost surely over the summarizer's randomness on both sides,*

$$g(u \oplus v) \sim g(z_u \oplus z_v),$$

and both sides determine $f^*(u \oplus v)$.

In plain language: if the children are already correct and both merge calls keep the oracle value of what they read, then summarizing the raw parent directly and summarizing the merged children land on the same oracle value. That value is the parent's true oracle value. This is the one inductive step the preservation proof below repeats at every internal node.

Proof. By hypothesis (i), $z_u \sim u$ and $z_v \sim v$. Context compatibility (Assumption 1) applied to the first equivalence, with context z_v concatenated on the right, gives $u \oplus z_v \sim z_u \oplus z_v$. Applied to the second equivalence, with context u concatenated on the left, it gives $u \oplus v \sim u \oplus z_v$. Chaining these,

$$u \oplus v \;\sim\; u \oplus z_v \;\sim\; z_u \oplus z_v.$$

By hypothesis (iii), $g(u \oplus v) \sim u \oplus v$ and $g(z_u \oplus z_v) \sim z_u \oplus z_v$. Combining with the displayed chain and transitivity of $\sim$,

$$g(u \oplus v) \;\sim\; u \oplus v \;\sim\; z_u \oplus z_v \;\sim\; g(z_u \oplus z_v),$$

so $g(u \oplus v) \sim g(z_u \oplus z_v)$. Since $\sim$ is defined by $d_{\mathcal{Y}}(f^*(\cdot), f^*(\cdot)) = 0$, every string in this equivalence chain shares the same oracle value, namely $f^*(u \oplus v)$. In particular both $g(u \oplus v)$ and $g(z_u \oplus z_v)$ determine it. □

B.3 Proof of Proposition 1 (Oracle-Consistency Propagation)

Proof. Fix a realized full binary tree T over document x, partitioned into leaves $b_1, \ldots, b_k$. For each node u, define the latent raw span $S(u)$ recursively: $S(u) = b_i$ when u is the leaf holding block b_i, and $S(u) = S(u_L) \oplus S(u_R)$ when u is an internal node with children (u_L, u_R). Define the stored summary s_u recursively to match the tree's own construction: $s_u = g(S(u))$ at leaves and $s_u = g(s_{u_L} \oplus s_{u_R})$ at internal nodes. Let r denote the root, so $S(r) = x$ and $s_r = s_{\text{root}}$. Assume C1 and C3 hold on every realized call of the tree, and assume Assumption 1.

We show $s_u \sim S(u)$ for every node u, by structural induction on node height.

Base case. If u is a leaf holding block b_i, then $S(u) = b_i$ and C1 gives $s_u = g(b_i) \sim b_i = S(u)$.

Inductive step. Let u be an internal node with children u_L, u_R, and suppose $s_{u_L} \sim S(u_L)$ and $s_{u_R} \sim S(u_R)$ (the induction hypothesis, established one level down). Apply Lemma 1 with $u \leftarrow S(u_L)$, $v \leftarrow S(u_R)$, $z_u \leftarrow s_{u_L}$, $z_v \leftarrow s_{u_R}$: hypothesis (i) is the induction hypothesis just stated, hypothesis (ii) is Assumption 1, and hypothesis (iii) is C3's first link, instantiated once at the raw span $S(u_L) \oplus S(u_R)$ and once at the realized merge call $g(s_{u_L} \oplus s_{u_R})$. The lemma gives

$$g(S(u_L) \oplus S(u_R)) \sim g(s_{u_L} \oplus s_{u_R}),$$

and by hypothesis (iii) the left side is itself oracle-consistent with $S(u_L) \oplus S(u_R) = S(u)$. Since $s_u = g(s_{u_L} \oplus s_{u_R})$ by construction, transitivity of $\sim$ gives $s_u \sim S(u)$.

By induction, $s_u \sim S(u)$ holds at every node, and in particular at the root: $s_{\text{root}} = s_r \sim S(r) = x$, i.e.

$$d_{\mathcal{Y}}(f^*(s_{\text{root}}), f^*(x)) = 0.$$

Idempotence extension. Suppose in addition C2 holds on every realized recompression call. Define a sequence of further passes on the root summary, $Z^{(1)} := s_{\text{root}}$ and $Z^{(R+1)} := g(Z^{(R)})$ for $R \geq 1$. Because $Z^{(1)} \in \text{range}(g)$ and g maps into its own range, every $Z^{(R)}$ for $R \geq 1$ lies in $\text{range}(g)$, so C2 applies at each step: $Z^{(R+1)} \sim Z^{(R)}$. We show $Z^{(R)} \sim x$ for all $R \geq 1$ by induction on R. The base case $R = 1$ is the root conclusion just proved, $Z^{(1)} = s_{\text{root}} \sim x$. For the inductive step, if $Z^{(R)} \sim x$ and $Z^{(R+1)} \sim Z^{(R)}$, transitivity of $\sim$ gives $Z^{(R+1)} \sim x$. Hence $d_{\mathcal{Y}}(f^*(Z^{(R)}), f^*(x)) = 0$ for every $R \geq 1$: oracle-consistency survives any number of additional summarization passes on the root. □

B.4 Context Compatibility

Assumption 1 is used exactly once, inside Lemma 1's proof, and that single use is unavoidable. The lemma needs to move from two separate child-level facts, $z_u \sim u$ and $z_v \sim v$, to one parent-level fact, $z_u \oplus z_v \sim u \oplus v$. Oracle-consistency is defined pointwise on whole strings, so $z_u \sim u$ says nothing by itself about what happens when z_u is embedded next to a sibling. The assumption is the substitution property that licenses the embedding.

Remark 1 (Two single-slot substitutions suffice)**.** *Assumption 1 is stated for one substitution inside a fixed context. Proving $z_u \oplus z_v \sim u \oplus v$ from $z_u \sim u$ and $z_v \sim v$ uses it twice: once holding z_v fixed as the right context while replacing u by z_u on the left, and once holding u fixed as the left context while replacing v by z_v on the right, chained through $\sim$'s transitivity. No simultaneous two-slot substitution property is needed.*

This is exactly the chain $u \oplus v \sim u \oplus z_v \sim z_u \oplus z_v$ used in the proof of Lemma 1: single-slot substitution applied twice, not a joint two-slot substitution assumed as a separate primitive. Restating it here makes the minimal use visible on its own.

Everything else is in the supplement: preference-learning transport, structured-error equivalence, gap bounds beyond the shape used in Section 4, invariance across merge orders, multi-round recompression beyond the extension proved above, and the C2-independence of the base preservation argument.

C Audit Estimation Details

Section 4 states the audit's load-bearing displays: the leaf violation rate (Equation 1), the Horvitz–Thompson estimator (Equation 2), and the accuracy bound they feed (Equation 3). Appendix C carries the estimation details behind them: the per-law estimators the audit runs, how their guarantees scale across a tree, and the sample-splitting protocol that keeps the audit valid when the tree itself is built from data. Clipped and Hajek estimator variants, a worst-case sampling-robustness bound, the audit reporting template, and the three-term bound decomposition are reported in the supplement.

C.1 Per-Stratum Estimators

Each law in Section 2 is audited by a different query on a different node type, so the audit is three separate finite-population estimation problems sharing one estimator. Write $U(x) = U_1(x) \cup U_2(x) \cup U_3(x)$ for the partition of x's realized nodes into leaves (C1 stratum), recompression calls (C2 stratum), and merge nodes (C3 stratum), with population sizes N_1, N_2, N_3.

C1: leaf sufficiency. For a sampled leaf block b_i, the query compares the oracle value of the leaf's summary against the oracle value of the raw span:

$$\hat{p}_{\mathrm{C1}} := \frac{1}{n_1} \sum_{i=1}^{n_1} \mathbb{I}\left\{ d_{\mathcal{Y}}(f^*(g(b_i)), f^*(b_i)) > \tau \right\},$$

where τ is a tolerance (usually 0). The corresponding distortion contribution to $\hat{\mu}$ is $Y_i = d_{\mathcal{Y}}(f^*(g(b_i)), f^*(b_i))$ itself, undiscretized.

C2: idempotence at recompression. For a sampled summary z_k that has already passed through g once, the query checks whether resummarizing it moves the oracle value:

$$\hat{p}_{\mathrm{C2}} := \frac{1}{n_2} \sum_{k=1}^{n_2} \mathbb{I}\left\{ d_{\mathcal{Y}}(f^*(g(z_k)), f^*(z_k)) > \tau \right\}.$$

This stratum is only populated when the deployed pipeline includes post-processing or cleanup rounds. A tree with no recompression calls has $N_2 = 0$ and reports no C2 rate.

C3: merge consistency. A merge node u with children u_L, u_R is audited by comparing the merged summary's oracle value against the oracle value of the span below it. Because the comparison can run against either the raw concatenation or the joint summary, the merge query has two forms: against the raw children,

$$\hat{p}_{\mathrm{C3,raw}} := \frac{1}{n_3} \sum_{j=1}^{n_3} \mathbb{I}\left\{ d_{\mathcal{Y}}(f^*(g(g(u_L^j) \oplus g(u_R^j))),\ f^*(u_L^j \oplus u_R^j)) > \tau \right\},$$

and against the joint summary of the concatenated children,

$$\hat{p}_{\text{C3,joint}} := \frac{1}{n_3}\sum_{j=1}^{n_3}\mathbb{I}\left\{d_{\mathcal{Y}}(f^*(g(g(u_L^j)\oplus g(u_R^j))),\ f^*(g(u_L^j\oplus u_R^j)))>\tau\right\}.$$

The raw-children form is available only near the leaves, where $u_L \oplus u_R$ still fits inside the judge's context window. At internal merges the children are already stored summaries, and only the joint form is available. Which form runs at each depth is fixed by the audit design, not by the law being checked.

Each stratum's violation rate is design-unbiased for its own finite-population violation probability under the sampling law used within that stratum. This is the equal-probability special case recorded in Appendix C.2 below, applied separately per stratum. The three rates are diagnostics: a low C1 rate paired with a high C3 rate says the leaves are fine and the merges are not, exactly as in the two-output framing of Section 4. Distortion means from the same three strata (the undiscretized Y_i values, not the thresholded indicators) feed $\hat{\mu}$ in Equation 2.

C.2 Equal-Probability Special Case

Equation 2 is stated for general logged inclusion probabilities π_i. When the audit design samples each stratum uniformly at random, π_i is constant within the stratum and the HT estimator collapses to a sample mean: for n_1 leaves drawn uniformly from N_1 candidates, $\hat{p}_{\text{C1}}$ and the corresponding distortion mean are ordinary averages over the drawn units, with no reweighting term. The same collapse applies stratum-by-stratum to C2 and C3. The per-stratum formulas above run this estimator. The general HT form in Equation 2 is needed only once inclusion probabilities vary within a stratum (for example, an audit that oversamples merges near the root).

C.3 Scaling Across a Tree

A realized tree with N leaves has $M = N - 1$ merges, so a root-level failure event can be reached through any of N leaf checks, M merge checks, or $R - 1$ recompression checks (for R cleanup rounds). A union bound over per-unit violation probabilities gives

$$\mathbb{P}[\text{root violation}] \le \sum_{i=1}^{N} p_{\text{C1},i} \;+\; \sum_{j=1}^{M} p_{\text{C3},j} \;+\; \sum_{r=1}^{R-1} p_{\text{C2},r}, \tag{6}$$

where each summand is the true violation probability of the corresponding checked event. Under homogeneous per-stratum upper bounds this reduces to $Np_{\text{C1}}^{\max} + Mp_{\text{C3}}^{\max} + (R-1)p_{\text{C2}}^{\max}$. Even a small per-node failure probability accumulates linearly in tree size, so the tolerable per-node rate shrinks as the tree grows. Equivalently, if a document has N leaves and leaf sufficiency failure rate p_{C1}, the probability that every leaf is correct is approximately $(1 - p_{\text{C1}})^N \approx e^{-Np_{\text{C1}}}$. Large N demands small local error rates for a reliable root summary.

The bridge from these per-stratum violation rates to the accuracy bound in Equation 3 is the distortion estimator $\hat{\mu}$. With the ideal oracle, HT applied to $Y_i = d_{\mathcal{Y}}(f^*(s_i), f^*(S(i)))$ is unbiased for the true audited-unit distortion mean, and Equation 6 decomposes that mean's failure probability across the same three strata used to compute it.

C.4 Honesty Under Adaptive Partitioning

The estimators above assume the audited population $U(x)$ and its logged inclusion probabilities do not depend on the sample used to evaluate them. That assumption breaks when the partitioner,

the summarizer, or the judge is adaptive. Each can be fit on data, and fitting opens an interference path between the tree's construction and the audit that checks it. An honesty protocol keeps the audit valid under adaptivity: adaptation runs on a train-role partition of the data, and the audit (with any calibration set feeding B_{cal}) runs only on the held-out role intersection. The sampled nodes under audit were then never used to fit the partitioner, summarizer, or judge that produced them. The learned-partition program of Section 7 builds on this sample-splitting protocol.

D Manifesto Document-Label Replication

Appendix D carries the evidence behind the document-label result of Sections 6.2 and 6.3: the corpus and expert benchmarks, the pipeline protocol, the full headline comparison, the leaf-size ablations, the robustness slices, and the call-count accounting that supports the replication claim. The supplement reports the alternating f/g prompt ladder that optimizes the scorer and summarizer against each other, the scoring-order serving note, and the oracle-grounded training path.

D.1 Corpus, Targets, and Expert Benchmarks

The main comparison uses the same Manifesto Project platform benchmark as Benoit et al. [2025]: 235 party election platforms from parliamentary democracies, filtered to election years for which the Benoit–Laver and CHES expert surveys provide party-position estimates. Benoit et al.'s broader paper also reports 23 Klüver coalition agreements. The headline comparison uses the Manifesto Project platform rows. Raw text comes from the Manifesto Corpus 2025a release [Lehmann et al., 2025b]. Where the release text carried promotional footer truncation, we used the full official Manifesto Project text.

The prediction target is an expert-survey mean. Benoit et al.'s replication archive provides one harmonized mean party-position estimate for each (manifesto, dimension) pair. Using that archive rather than recomputing from raw CHES plus Benoit–Laver avoids rescaling, party-year-join, and missing-value discrepancies. As a check, we recomputed Benoit's published Figure 1 correlations from their released LLM-score and expert-mean tables and matched their numbers to within 0.005 on every dimension.

Benoit et al.'s Table 3 reference is a split-expert agreement benchmark, not a sample-specific limit for our held-out split. In the Benoit–Laver replication data, experts within each country are randomly divided into two groups. Each group produces a mean party-position estimate on the same issue-position scale, and the two vectors of mean estimates are correlated. The economic value used below is $r = 0.880$. The macro value across the six policy dimensions is 0.873.

D.2 Pipeline Protocol

Every row in Table 6 corresponds to a single pipeline configuration with three components. The leaf summarizer compresses each text span under a dimension-specific preservation rubric; the prompt targets Benoit's 300–400 word summary length but allows longer outputs when aggressive compression would drop policy-relevant evidence. The merge summarizer combines adjacent child summaries under the same rubric, with the same information-preservation target, and may carry both child summaries forward when that is the least lossy state. The scorer maps the root summary to the seven-point policy scale under the Benoit rubric for dimension d, recording a missing response only when the model cannot produce a valid score.

The six scoring contexts are the exact prompts released in Benoit's replication materials, so the scorer sees the same rubric text as the GPT-4o scorer in their pipeline. The wording matters:

swapping an earlier paraphrased rubric for the exact released text shifted macro correlation by +0.008 on held-out summaries.

Leaf partitioning. Leaves are produced by paragraph-boundary splitting inside a character budget. Manifestos on the matched roster average 168K characters, with the shortest platforms in the 20–60K range. At $c = 8$K a mean-length manifesto yields ≈21 leaves and a short one 3–8; at $c = 24$K the counts drop to ≈7 and 1–3.

Model. All runs use GEMMA-4-31B-IT-NVFP4 served from a local GPU node with a 12K-token context cap. The cap easily holds the summaries used here (∼3K tokens) and keeps the inference budget fixed across settings.

D.3 Headline Comparison

Table 6 is the full version of Table 3: every macro correlation reported in the main text, plus the two exact-model and scorer-only baselines that motivate the C-Tree comparison.

Method	Macro r	Calls/manifesto-dim.	Model class
Split-expert reference	0.873	—	human coders
Proprietary ensemble (18 scores)	0.817	18	3 frontier LLMs
Gemma-3-27B-IT replication	0.793	1	open-weight
Scorer-only (Benoit summaries, no tree)	0.743	1	open-weight
Full-document read, 48K-char. truncation	0.771	1	open-weight
Leaf summaries concatenated, no merge, $c = 8$K	0.800	≈40	open-weight
C-Tree, $c = 8$K leaves	0.787	≈45	open-weight
C-Tree, $c = 4$K leaves	0.783	≈90	open-weight

Table 6: Full headline comparison. Macro Pearson r against Benoit et al. expert-survey means, averaged over six policy dimensions. The five lower rows share one campaign on the full matched roster (one model, one decoding configuration). Calls/manifesto-dim. counts language-model calls per manifesto-dimension pair, measured from run artifacts; the ensemble figure is the published protocol's per-pair call count. The scorer-only row runs only the scoring stage on Benoit's own anonymized summaries (no tree).

The complete grid covers the per-dimension C-Tree at $c \in \{2, 4, 8, 16, 24, 32, 64\}$K and the joint C-Tree at $c \in \{8, 16, 24, 32, 64\}$K. Table 7 collects the baselines and ablations that surround it.

Three of these rows carry a claim. Decoding choices left the ranking of the main configurations essentially unchanged. The Gemma-3 exact-model gap to Benoit et al.'s reported 0.793 is not explained by rubric wording (+0.008 delta) or output-format wrapping (−0.003 delta); we attribute the residual to model-weight revision drift between the Benoit vintage and our local checkpoint. And the scorer-only row prices the scoring stage alone: the jump from 0.743 to the 0.77–0.80 band in Table 6 comes from the summarization step, not the scorer.

D.4 Leaf-Size Invariance Ablations

Two ablations test the leaf-size-invariance prediction of Section 6.2: when the local laws hold, the root score should not depend on how the document is cut.

Variant	Macro r	Campaign
Flat read, 48K-char. truncation (no tree)	0.771	full roster
Concat, no merge, $c = 8$K	0.800	full roster
Scorer-only (Benoit summaries, no tree)	0.743	full roster
Decoding variants, per-dimension C-Tree	within 0.009 of central	frozen sweep
Decoding variants, joint C-Tree	within 0.008 of central	frozen sweep
Gemma-3-27B exact-model replication	0.723–0.731	released summaries

Table 7: Baselines and ablations around the headline grid. Macro Pearson r against expert-survey means. The flat row is the direct long-context control (raw text truncated and scored, no tree); the concat row runs leaf summaries with no merges; the scorer-only row runs the scoring stage alone on Benoit et al.'s released summaries (0.73–0.75 across frozen prompt variants). The decoding rows report the spread of each variant across low- and medium-temperature runs, with and without three-sample aggregation, on the frozen sweep campaign. The exact-model row scores Benoit's released summaries with the Gemma-3-27B BF16 checkpoint of their Table 6.

Leaf-size invariance. We ran the per-dimension C-Tree at $c \in \{4, 8, 16, 24, 32, 64\}$K on the same manifestos; Figure 3 plots the sweep per dimension. Macro r moved by 0.027 across leaf sizes. The five non-decentralization dimensions stay in the same high-correlation regime; decentralization is the exception, discussed next.

The dimension where leaf-size invariance breaks is the dimension the published literature flags. Decentralization's Pearson r improves monotonically as leaves shrink: 0.489 at $c = 64$K, 0.543 at $c = 8$K, 0.584 at $c = 2$K (Figure 3, bottom row). Benoit et al. [2025] flag decentralization as a boundary case in their Figures 2–3, where manifesto quasi-sentence coders and expert-survey respondents disagree about what resolves a party's position. Their own Table 3 split-expert reference on decentralization is 0.78, well below the ≥ 0.88 references for the other five dimensions. The smaller-leaf regime forces the tree to carry the disambiguating evidence across merges rather than summarizing it away at the leaf, and the correlation improves as a result. When the oracle cares about between-leaf interactions, finer leaves help.

D.5 Additional Robustness Diagnostics

Two slices check that no subpopulation carries the headline result: document length and election era. Table 8 reports both, from the same full-roster headline run as Table 6. The longest documents sit inside the headline result, and no single era carries it; decentralization stays the weak dimension in every slice, matching its corpus-wide pattern.

Secondary Manifesto Project coded-score target. Benoit et al. also discuss Manifesto Project quasi-sentence counts transformed into coded logit scores. We treat those as a construct-validity check, not as the target benchmark here, because they measure a different object from expert-survey party-position means. Against the coded-score target, the tree, flat, concat, and joint variants produce macro correlations in the 0.337–0.397 range, and the European integration correlations are strongly negative. The expert-survey target and the coded-count target are not interchangeable validation labels.

Dimension	Manifestos > 100K chars	Range across eras
Economic	0.89	0.854–0.904
Social	0.84	0.783–0.857
Immigration	0.85	0.787–0.872
European integration	0.90	0.900–0.917
Environment	0.83	0.789–0.820
Decentralization	0.40	0.355–0.480

Table 8: Robustness slices, per-dimension C-Tree at $c = 8$K on the full matched roster. The long-document column restricts to manifestos longer than 100K characters ($n = 92$–118 per dimension). The era column gives the minimum and maximum Pearson r across election-era buckets (1989–1999, 2000–2009, 2010–2019); immigration and European integration have no 1989–1999 cases in the benchmark and span the two later eras.

D.6 Call-Count Accounting

The call counts decompose by pipeline variant and LLM call type (Table 9). The ensemble's count is fixed by its published protocol: 3 LLMs × 2 shot-settings × 3 summaries = 18 calls per manifesto–dimension pair, nine summarization and nine scoring. A tree's count follows from the document length: $2\lceil n_{\text{chars}}/c \rceil - 1$ summarize and merge calls per manifesto plus its scoring calls, and the matched roster averages 168K characters per manifesto, so at $c = 8$K the joint variant makes ≈ 47 calls per manifesto.

Pipeline	Calls/manifesto-dim.	Corpus total	Model routing
Proprietary ensemble	18	25,380	3 frontier APIs
Joint C-Tree, $c = 8$K	≈ 8	$\approx$ 11,000	one local open-weight
Per-dimension C-Tree, $c = 8$K	≈ 45	55,152 ($\approx$ 64,000 proj.)	one local open-weight

Table 9: Measured call accounting over the 235-platform benchmark. Calls per manifesto–dimension pair and corpus totals; the ensemble row is the published protocol's count (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro), tree rows are measured from run artifacts. The per-dimension corpus total is measured on the realized rosters and projected to the full roster.

The accounting supports the replication claim with one asymmetry worth stating plainly. The joint variant reproduces the measurement in fewer than half the published protocol's calls ($\approx$ 11,000 against 25,380). The per-dimension variant, the most accurate local configuration, makes about 2.3× the ensemble's total calls. Every call in either variant goes to one locally served open-weight model, with no frontier API in the loop.

D.7 Reproduction

The full sweep regenerates the tables above from per-manifesto predictions in four protocol steps: evaluate the open-weight model; run the per-dimension pipeline over the six Benoit dimensions and leaf sizes $c \in \{4, 8, 16, 24, 32, 64\}$K; run the joint two-pass pipeline over $c \in \{8, 16, 24, 32, 64\}$K; and aggregate the resulting manifesto-level scores against Benoit's expert means. The aggregation unit is always the manifesto–dimension prediction, so the reported correlations do not depend on the order in which predictions are produced.

E Sentence-Leaf Node-Label Details

Appendix E carries the evidence behind the node-label results of Sections 6.4 and 6.5: quasi-sentence codes supply the labels, sentences supply the leaves. It gives the label construction, the rollup validation and construct ceiling, the training protocol, the full sentence-leaf ladder, the dense/sparse per-dimension breakdown, the expert-dimension grid, and the census-validated audit. The alternating prompt ladder, gold seeding, LLM merge collapse under prompt search, and per-dimension hyperparameter optimization are reported in the supplement.

E.1 Label Construction

The manifesto measurement problem has two layers of ground truth. Document-level labels are the pair used in the document-label replication of Appendix D: the Benoit et al. [2025] expert-survey means on six seven-point policy scales, available for the 235-platform benchmark, and the Manifesto Project's own published MPDS row [Lehmann et al., 2025b], one electoral program per row, with total coded quasi-sentence counts and per-category shares. For every annotated platform used below, the Corpus quasi-sentence rows link back to this document row.

The Manifesto Corpus release [Lehmann et al., 2025a] provides the underlying per-quasi-sentence annotations. Each coded quasi-sentence (a clause-level unit, roughly one policy statement per unit) carries one category code from a 56-category Comparative Manifesto Project (CMP) scheme, assigned by a trained human coder under the project's handbook. In the release slice used here, those annotations cover 2,157 platforms ($\sim$2.27M coded spans). The hand labels stop there. No coder labels a C-Tree merge. Because the codes are additive counts, they induce an exact oracle value at every node of every tree: each leaf, each internal merge span, and the root. No oracle redesign is needed.

RILE, the left-right index, is a deterministic function of the CMP category counts: with R the count of quasi-sentences bearing one of 13 designated right codes, L the count bearing one of 13 left codes [Laver and Budge, 1992], and N the total count of coded non-header quasi-sentences,

$$\text{RILE} \;=\; 100 \cdot \frac{R - L}{N}, \qquad \text{RILE}_{\text{norm}} \;=\; \frac{\text{RILE} + 100}{200} \in [0, 1]. \tag{7}$$

RILE is the signed share of coded statements that lean right, so it inherits the additivity of the counts it is built from. The seven domain shares $\text{dom}_1, \ldots, \text{dom}_7$ are the analogous count ratios for the seven CMP handbook policy domains (external relations, freedom and democracy, political system, economy, welfare and quality of life, fabric of society, social groups).

For any contiguous span x of a platform, aggregating the codes of the quasi-sentences inside x and applying Equation (7) yields an exact oracle value $f^*(x)$: an 8-vector $(\text{RILE}_{\text{norm}}, \text{dom}_1, \ldots, \text{dom}_7)$, the *compact state*. Because the aggregation is a count sum, the oracle is exactly mergeable: the compact state of a parent span is a known function of its children's counts. The two children and the parent inherit targets from the same quasi-sentence table, making the merge triangle observable on real political text. Every node of every C-Tree over a coded platform has an exact target. This is the structure the controlled settings of earlier sections relied on and that generic long-document corpora lack.

E.2 Rollup Validation

The bottom of this label system should reproduce the top. Rolling the quasi-sentence codes up to the document level and correlating with the published MPDS document scores across the 2,157 coded platforms gives Pearson $r = 0.9975$ for RILE and $r \in [0.924, 0.999]$ across the seven domain

shares (Table 10, left). The residual is bookkeeping, not noise. The induced rollup uses the non-header quasi-sentence count as its denominator and folds handbook-v5 four-digit subcodes into their three-digit parents. The published per-category percentages use MPDS's own denominator conventions. The published document-level label is, to numerical precision, the additive rollup of the induced quasi-sentence labels. A method that recovers the right local states and merges them lawfully recovers the published document score.

The same check against the Benoit et al. [2025] expert-survey means gives a much lower ceiling. Ordinary least squares from the eight compact-state features to the expert mean reaches five-fold cross-validated R^2 between 0.035 (decentralization) and 0.472 (immigration) (Table 10, right). The expert-survey construct is a holistic judgment the additive category counts only partly determine. This is the *construct ceiling*: it bounds what any method trained on the quasi-sentence codes alone can reach against the external target, and it is set by the label system, not by the model. The *internal* target (the rollup of the human-coded quasi-sentences) is fully decomposable and supervisable at every node. The *external* target (expert-survey means) requires the model to read text, and the quasi-sentence codes alone cannot reach it.

Internal rollup validity

Compact dimension	Pearson r
RILE	0.9975
domain_1 (external relations)	0.972
domain_2 (freedom/democracy)	0.998
domain_3 (political system)	0.996
domain_4 (economy)	0.997
domain_5 (welfare)	0.999
domain_6 (fabric of society)	0.994
domain_7 (social groups)	0.924

External construct ceiling

Expert dimension	CV R^2
decentralization	0.035
European integration	0.181
environment	0.278
social	0.380
economic	0.397
immigration	0.472

Table 10: Left: internal rollup validity. Pearson correlation between the rollup of the quasi-sentence codes and the published MPDS document score, per compact dimension (n = 2,157 platforms). Right: external construct ceiling. Five-fold cross-validated R^2 of ordinary least squares from the eight compact-state features to each Benoit expert-survey mean. The rollup identifies the internal target almost exactly; the same features only partly determine the external construct, and the gap is set by the label system, not the model.

E.3 Bundles, Splits, and Training Protocol

The experiments run on frozen *labeled-tree bundles*. The leaf unit is the sentence, the smallest readable span of the text: a quasi-sentence never crosses a sentence boundary, so consecutive quasi-sentences group deterministically into sentences, and a sentence's label is the count aggregate of its quasi-sentences' codes (most sentences contain exactly one quasi-sentence; the corpus mean is 1.39). A bundle fixes a leaf size of $\ell \in \{1, 2, 4, 8, 16\}$ sentences per leaf, partitions each platform into contiguous leaves of ℓ sentences, builds the binary merge tree, and stamps every node with its exact compact state via the count aggregation above. No LLM is involved in labeling, and no merge label is collected by hand. The parent target is the count rollup of the quasi-sentences under that parent. The corpus is 217 Western European platforms under a fixed split (139 train, 30 validation, 48 test) shared across all leaf sizes, so a leaf-size comparison never mixes documents. One platform was excluded from the original 218: its text lacks terminal punctuation, so the deterministic grouping produced degenerate sentences hundreds of quasi-sentences long, and the roster was pinned before

the rebuild rather than resampled. Leaf size changes the amount of supervision by an order of magnitude: the $\ell = 1$ bundle has about 313,000 labeled nodes, the $\ell = 16$ bundle about 20,000.

The implementation is the `treepo` package. Its public entry point is `treepo.fit`, which takes a declarative spec and runs an alternating f/g ladder for any *family*: an object supplying its own parameterization of the summarizer g and readout f behind a fixed four-method contract. The surrounding loop (alternation, node sampling, metrics, audit logging) is family-agnostic. The results here use the neural-operator family (`fno`): each leaf is embedded once by a small embedding model (EMBEDDINGGEMMA-300M, one 2,048-token chunk per leaf, with the rare overlong leaf truncated under a logged count), and f and g are Fourier Neural Operator stacks (12 epochs, batch 16, learning rate 2.37×10^{-3}) trained by gradient descent on node-level losses. Embeddings are computed once per bundle and cached to disk. The FNO training itself takes seconds per cell. The LLM prompt-program family counterpart, its label-echo and merge-collapse failure modes, and the mean-over-dimensions reward diagnosis are reported in the supplement.

The alternation is the two-stage schedule of Section 5.5, f first ($f^1g^1 \to f^2g^1 \to f^2g^2$). The f-stage row isolates the readout trained on leaf states with an untrained merge. The g-stage row adds the merge trained against the current readout. Metrics follow the internal/external split above. *Internal* Pearson and MAE compare root predictions to the internal rollup (the exact compact state of the root). *External* Pearson compares them to the published MPDS document score. All correlations are per-dimension. Pooling the eight dimensions into one correlation inflates the number through between-dimension mean separation: the pooled statistic rewards a model for predicting that economy is discussed more than decentralization, which is not measurement.

E.4 Full Leaf-Size Ladder (Neural-Operator Family, RILE)

Table 11 is the full version of Table 4: Pearson and MAE at each stage against the published MPDS RILE (normalized), test split, $n = 48$.

leaf ℓ (sentences)	stage	ext. r	ext. MAE
1	f^2g^1	0.459	0.083
1	f^2g^2	0.579	0.057
2	f^2g^1	0.582	0.078
2	f^2g^2	**0.713**	**0.051**
4	f^2g^1	0.620	0.083
4	f^2g^2	0.699	0.053
8	f^2g^1	0.625	0.064
8	f^2g^2	0.647	0.061
16	f^2g^1	0.631	0.072
16	f^2g^2	0.664	0.061

Table 11: Full leaf-size ladder: neural-operator family, RILE target, sentence leaves, test split ($n = 48$), single seed. Rows pair a leaf size ℓ with a training stage (Section 5.5). Scores are against the published MPDS RILE (normalized); the internal-rollup scores coincide to reported precision, the rollup validation of Appendix E.2 showing up as a consistency check. Bold marks the grid optimum, the trained merge at $\ell = 2$. The untrained f^1g^1 rows (all $r = 0$, constant prediction 0.5) are omitted.

The grid shows three things. First, the trained merge pays at every leaf size: the g-stage beats the f-stage in all five rows, with the largest lifts at the smallest leaves ($0.459 \to 0.579$ at $\ell = 1$, $0.582 \to 0.713$ at $\ell = 2$), where the merge chain is deepest and generic averaging loses the most. Second, the profile is non-monotone in ℓ: smaller leaves give more supervised nodes ($\ell = 1$ has about 313K) and deeper merge chains for a trained operator, and at this fixed training budget the trade resolves at a small intermediate leaf, $\ell = 2$. Third, the MAE column tracks the Pearson column cell for cell, so the peak is a genuine accuracy optimum rather than a correlation artifact.

The prompt-only pipeline of Appendix D does not face this tension: its summarizer is fixed and already competent, with no merge chain to keep lawful by training. Its leaf-size invariance (macro spread 0.027 over a 16× sweep) is the behavior of that fixed summarizer. The trained substrate's leaf-size profile is the cost of learning the summarizer from local supervision alone.

E.5 Dense/Sparse Per-Dimension Breakdown

The merge composes on dense dimensions and collapses on sparse ones, and the split tracks label density. Running the neural-operator family on the $\ell = 8$ sentence bundle, three seeds per dimension, and scoring each compact dimension separately gives Table 12 (f- and g-stage Pearson against the published MPDS share).

dimension	mean share	zero leaves	FNO f	FNO g
RILE	0.43	0.10	0.63	**0.65**
domain_5 (welfare)	0.32	0.42	0.58 ± 0.02	$\mathbf{0.62 \pm 0.00}$
domain_4 (economy)	0.25	0.51	0.64 ± 0.02	$\mathbf{0.67 \pm 0.01}$
domain_6 (fabric of society)	0.11	0.72	0.42 ± 0.25	0.42 ± 0.25
domain_3 (political system)	0.09	0.73	-0.05 ± 0.25	0.00 ± 0.23
domain_7 (social groups)	0.09	0.75	-0.02 ± 0.15	-0.02 ± 0.15
domain_2 (freedom/democracy)	0.07	0.80	0.05 ± 0.12	0.05 ± 0.12
domain_1 (external relations)	0.07	0.86	0.13 ± 0.23	0.13 ± 0.23

Table 12: Per-dimension Pearson against the published MPDS share, neural-operator family, $\ell = 8$ sentence bundle, test split ($n = 48$), mean ± sample sd over three seeds (RILE: single seed, from the ladder). "Mean share" is the dimension's mean coded share over leaves; "zero leaves" is the fraction of leaves carrying a zero share. The rules separate the dense dimensions (top), a seed-unstable boundary case (middle), and the sparse dimensions (bottom): dense dimensions compose, sparse ones collapse to a zero predictor. The LLM prompt-program family's numbers, and the label-echo confound that inflates its leaf stage, appear in the supplement.

On the dense dimensions (mean coded share above 0.25: RILE, economy, welfare) the neural-operator family learns the leaf readout and composes it through the learned merge, with seed spreads at or below 0.02. Fabric of society sits at the boundary (share 0.11) and is seed-unstable: one seed composes (0.69), two collapse (0.19–0.37). On the four sparse dimensions (73–86% of leaves carry a zero share) the readout collapses to predicting zero, and the f- and g-stage numbers are identical seed for seed: the merge inherits the collapse rather than causing it. The failure is legible. It is a direct function of how often the dimension appears, not a substrate limitation.

E.6 Expert-Dimension Grid and Winning Hyperparameters

The expert-dimension reconstruction of Section 6.5 uses the neural-operator family at a sixteen-quasi-sentence leaf (this campaign predates the sentence-axis grids and is unaffected by them) with LLM-generated span supervision at internal nodes, scored against the Benoit et al. [2025] expert means on the test split, three seeds per dimension. Hyperparameters were selected by an HPO sweep on the economic dimension only (best configuration: 384 Fourier modes, hidden width 32, two layers, head width 256, 12 epochs, learning rate 2.4×10^{-3}, loss weights leaf 1.57 / merge 0.89 / root 3.08) and transferred to the other five dimensions with no per-dimension tuning.

dimension	g-stage r (mean $\pm$ sd)	f-stage r	n	composition
economic	$\mathbf{0.734 \pm 0.016}$	0.671	50	lift
environment	0.632 ± 0.034	0.607	39	flat
social	0.527 ± 0.064	0.493	50	flat
immigration	0.440 ± 0.066	0.443	49	none
decentralization	0.271 ± 0.009	0.263	50	flat
EU	0.169 ± 0.010	0.177	49	none

Table 13: Neural-operator family scored against the Benoit expert-survey means, $\ell = 16$, test split, three seeds per dimension. The composition column marks how the learned merge moves the f-stage number: lift when the gain exceeds the seed spread, flat when it sits inside it, none when the merge gives the number back unchanged or below. The full-document frontier-LLM read (Appendix D) sits near $r = 0.86$ on the same task. The dimension ordering tracks the construct ceiling of Appendix E.2.

The ordering in Table 13 matches the CMP-code construct ceiling of Appendix E.2 (cross-validated R^2 from code counts: decentralization 0.03, EU 0.18, up to economic ~ 0.40), with one caveat each way. Immigration's mean is soft (seed spread 0.37–0.49). Decentralization has full expert coverage yet still floors, so the floor is signal, not sample size. Economic, the strongest dimension, is also the dimension the recipe was tuned on. Per-dimension HPO studies exist for the four weaker dimensions (about 50 Optuna trials each), with best validation objectives of 0.554 for immigration, 0.615 for environment, 0.540 for EU, and 0.679 for decentralization. Social has no per-dimension study. These are validation objectives, not test results. The decentralization gap (0.679 validation against the 0.271 transferred-recipe test r) is wide enough that validation overfitting is plausible. Table 13 therefore reports the transferred economic recipe as the result of record.

E.7 Census-Validated Audit

The manifesto corpus supplies what the audit of Section 4 otherwise has to approximate under a sampling budget: an exhaustive population of realized nodes, each carrying an exact induced oracle value (Appendix E.1). Where node labels are exhaustive, checking oracle-consistency at every node is a census. The census checks the sampled-audit estimator itself, on data where the ground truth the estimator recovers is computable rather than assumed.

Population. The realized nodes of the coded-manifesto trees built as in Appendix E.3: every leaf, internal merge, and root of every C-Tree in the sentence bundle, at a fixed leaf of eight sentences. The census uses the full 218-platform grid; the single-platform exclusion above affects only the trained bundles, since the census never embeds text. Each node carries its exact induced compact

state (Equation 7 and its domain-share analogues), so the population's true violation rate for any local law is a computable census quantity, not an estimate.

Design. Two arms share the same sampled node set at budgets $n \in \{50, 200, 800\}$, drawn with logged inclusion probabilities π_i under the sampling design of Section 4. The first arm scores sampled nodes against their exact compact state, forms the Horvitz–Thompson estimate of the population violation rate with its envelope (Equation 3), and compares both against the census truth computed over the full node population. The second arm scores the same sampled nodes with an LLM judge in place of the exact compact state. The gap between the two arms measures the calibration residual between judge-based and oracle-grounded auditing, on data where the oracle-grounded number is exact.

Results. Table 14 reports the merge-consistency (C3b) violation rate at each budget, over twenty independent replications. The estimate tightens onto the census truth as the budget grows, the envelope covered the truth in every replication at every budget, and the judge arm's calibration residual is below 0.01 throughout. The mean-distortion estimate behaves the same way (0.0049 ± 0.0004 against a census truth of 0.0050 at $n = 800$), as do the per-dimension estimates (RILE and the seven domain shares; per-budget JSONs in the run artifacts).

budget n	HT estimate	envelope width	census truth	judge-arm residual
50	0.163 ± 0.053	0.478	0.140	0.009
200	0.143 ± 0.030	0.219	0.140	0.008
800	0.139 ± 0.011	0.088	0.140	0.008

Table 14: Census-validated audit of the merge-consistency (C3b) violation rate. HT estimate is mean ± sd over twenty replications at budget n; envelope width is the mean empirical-Bernstein envelope; census truth is the exact population violation rate computed over every realized node; judge-arm residual is the gap between the LLM-judge-scored estimate and the exact-scored estimate at the same budget (B_{cal}). Envelope coverage of the census truth was 20/20 at every budget.

References

Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. Mergeable summaries. *ACM Transactions on Database Systems*, 38(4):28:1–28:40, 2013. doi: 10.1145/2500128.

Susan Athey and Guido Imbens. Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27):7353–7360, 2016.

Kenneth Benoit, Scott De Marchi, Conor Laver, Michael Laver, and Jinshuai Ma. Using large language models to analyze political texts through natural language understanding. *American Journal of Political Science*, 2025. doi: 10.1111/ajps.70050. URL https://onlinelibrary.wiley.com/doi/10.1111/ajps.70050.

David Blackwell. Equivalent comparisons of experiments. *Annals of Mathematical Statistics*, 24(2): 265–272, 1953.

Naoki Egami, Musashi Hinck, Brandon M. Stewart, and Hanying Wei. Using imperfect surrogates for downstream inference: Design-based supervised learning for social science applications of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Ronald A. Fisher. On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society of London. Series A*, 222:309–368, 1922.

D. G. Horvitz and D. J. Thompson. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952. doi: 10.1080/01621459.1952.10483446.

Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

Michael Laver and Ian Budge, editors. *Party Policy and Government Coalitions*. St. Martin's Press, New York, 1992.

Pola Lehmann, Simon Franzmann, Denise Al-Gaddooa, Tobias Burst, Christoph Ivanusch, Jirka Lewandowski, Sven Regel, Felicia Riethmüller, and Lisa Zehnter. Manifesto corpus. version 2025-1, 2025a. URL https://manifesto-project.wzb.eu/. Berlin: Wissenschaftszentrum Berlin für Sozialforschung (WZB) / Göttingen: Institut für Demokratieforschung (IfDem).

Pola Lehmann, Simon Franzmann, Denise Al-Gaddooa, Tobias Burst, Christoph Ivanusch, Sven Regel, Felicia Riethmüller, Andrea Volkens, Bernhard Weßels, and Lisa Zehnter. The manifesto data collection. manifesto project (mrg / cmp / marpor). version 2025a, 2025b. URL https://manifesto-project.wzb.eu/. Berlin: Wissenschaftszentrum Berlin für Sozialforschung (WZB) / Göttingen: Institut für Demokratieforschung (IfDem).

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

Nicolas Merz, Sven Regel, and Jirka Lewandowski. The Manifesto Corpus: A new resource for research on political parties and quantitative text analysis. *Research & Politics*, 3(2), 2016.

Stefan Wager and Susan Athey. Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523):1228–1242, 2018.